Mechanistic Analysis of Over-Squashing and Attention Dilution in Hybrid Graph Neural Networks

By

Collin McDevitt

Dhawal Dixit

Andy Jiang

Supervisor: Dr. Greg Green

A Capstone Project

Submitted to the University of Chicago in partial fulfillment

of the requirements for the degree of

Master of Science in Applied Data Science

Division of Physical Sciences

November 2026

Abstract

Graph neural networks predict a property of each node by passing information along the edges of a graph. They can fail when the information a node needs sits far away. Message passing squeezes distant signals through narrow bridges in the graph (over-squashing), while global attention spreads its focus over too many nodes (attention dilution). This project asks whether the structure of a graph and the range of a task predict how far a model should be allowed to look. We study large social graphs from SNAP (Twitch, Reddit and the Twitter Higgs-boson network), which existing long-range benchmarks do not cover. We measure over-squashing with excess effective resistance and Jacobian sensitivity, dilution with attention entropy, and task range with a Jacobian-based measure. Our first knob, a blend weight between message passing and attention, was partly undone by learnable normalisation scales, so we replace it with an attention radius k. On Twitter Higgs, no graph model beat a features-only baseline when trained on 500-node subgraphs, and a simple check showed that cutting the graph threw away most of the neighbour signal. Full-graph training and planted targets with a known range are running.

Keywords: graph neural networks, social networks, over-squashing, attention dilution, effective resistance, attention radius, task range, graph transformers, semi-synthetic benchmarks

Executive Summary

Many prediction problems live on networks: will this user become active, is this channel for adults, what does this community talk about. Graph neural networks (GNNs) answer such questions by letting each node collect information from its neighbours. Two designs dominate. Message passing relays information one hop at a time and can lose distant information at bottlenecks. Attention lets a node look at every other node at once, but it can then spread its focus too thin. Practitioners usually train both and keep the winner. We want a rule that says in advance how far a model should look, based on the graph's structure and on how far the task needs to reach.

What is done. We built a tested data pipeline for four large social graphs from SNAP, with checks that no future information leaks into the inputs. We built the models (message passing, three kinds of attention, hybrids and two baselines) and the measurements of over-squashing, dilution and range. We ran two sweeps on the Twitter Higgs graph.

What we learned. First, on large social graphs the standard bottleneck measure, effective resistance, mostly restates node degree, so we use a degree-corrected version (excess resistance). Second, our original knob, a blend weight α between message passing and attention, was partly undone during training by the learnable scales of the normalisation layers. We fixed this and added a cleaner knob, the attention radius k. Third, on Higgs no graph model beat a simple model that ignores the graph (test AUROC 0.710 for the simple model on 500-node subgraphs). A direct check explained why: cutting the graph into small pieces for training kept only about a third of the edges and removed most of the useful neighbour signal. On social graphs, subgraph training throws away what the models need.

What is pending. We are now training on the whole graph, sweeping both knobs. We are also building planted targets: real graphs and features with an added signal whose range is known exactly. They let us check that the range measure works and that the best radius follows the task's range. Results of these runs, the phase diagram, a test of graph foundation models and a benchmark release are still to come.

Table of Contents

Introduction . . . 1
- Problem Statement . . . 1
- Research Goals . . . 2
- Scope . . . 2

Background . . . 3
- The Intuition Behind the Tasks . . . 3
- Literature Review . . . 7

Data . . . 8
- Data Sources . . . 8
- Descriptive Analysis . . . 10

Methodology . . . 11
- Feature Engineering and Leakage Controls . . . 11
- Planted Targets with a Known Range . . . 12
- Modeling Framework . . . 13
- Training and Evaluation . . . 14
- Mechanistic Measurements . . . 16
- Experimental Plan . . . 17

Findings . . . 18
- Structure of the Social Graphs . . . 18
- Higgs: First α Sweep . . . 20
- Higgs: Affine-Free α Sweep . . . 21
- Why the Graph Did Not Help: Subgraph Partitioning . . . 23
- Full-Graph α and Radius Sweeps . . . 24
- Planted Long-Range Targets . . . 25
- Attention Radius on Subgraphs . . . 27

Discussion . . . 29
- What the Results So Far Mean . . . 29
- Limitations . . . 29
- How the Remaining Phases Answer the Thesis Question . . . 30

Conclusion . . . 31

References . . . 32

Appendix A: Background Experiments . . . 36
- River Discharge Forecasting . . . 37
- Controlled Tree Experiment . . . 41
- Pilot Results on Peptides-func and Reddit . . . 42
- Calibration of the Rebuilt Peptides Pipeline . . . 43
- Behaviour of the Measures on Untrained Models . . . 44

Appendix B: Mechanism Checks on Untrained Models . . . 45

Appendix C: Pilot Mechanistic Results . . . 46

Appendix D: Model Configurations . . . 47

Appendix E: Social-Graph Model and Run Configurations . . . 48

List of Figures

Figure 1. Two pictures behind the project . . . 4

Figure 2. Test AUROC against α on Higgs, affine-free sweep, with MLP and GCN reference lines . . . 22
Figure A1. Upstream growth profiles $V(r)$ of the river gauge networks . . . 37
Figure A2. Held-out test NRMSE against α for each outlet, weather inputs (top) and weather plus flow (bottom) . . . 39
Figure A3. Two-hop upstream volume against gain of the $\alpha = 0.75$ blend, held-out outlets, weather inputs, 3-day horizon . . . 40
Figure A4. Training cost against α for the river models . . . 41
Figure A5. Pilot Reddit test MSE by hop distance from the anchor, GraphSAGE against the virtual-master-node model (100 seeds) . . . 43
Figure A6. Gate G0 calibration run: training loss and validation AP of the rebuilt four-layer hybrid ($\alpha = 0.5$, seed 0) . . . 44
Figure B1. Jacobian sensitivity against hop distance for untrained local, hybrid and global models . . . 45
Figure B2. Attention entropy and long-range sensitivity against graph size for untrained attention models . . . 45
Figure C1. Pilot Jacobian sensitivity against effective resistance by architecture . . . 46
Figure C2. Pilot raw and normalised attention entropy against graph size . . . 47

List of Tables

Table 1. Social graphs and real targets . . . 9
Table 2. Structure of the social graphs (mean over test subgraphs) . . . 10
Table 3. Model variants . . . 15
Table 4. Experimental phases and status . . . 17
Table 5. Status of the social-graph experiments . . . 19
Table 6. Structure of the social graphs (means over test subgraphs) . . . 20
Table 7. Higgs `active_after`, first sweep: test AUROC (2,000-node subgraphs, 3 seeds) . . . 21
Table 8. Higgs affine-free α sweep: test AUROC (500-node subgraphs) . . . 22
Table 9. Neighbour signal with and without cutting the Higgs follow graph (logistic regression, test AUROC) . . . 24
Table 10. Neighbourhood growth and planted-target checks on the Higgs follow graph 27
Table 11. Attention radius on 500-node Higgs subgraphs: test AUROC . . . 28
Table A1. Status of the background experiments . . . 36
Table A2. Held-out river test NRMSE by configuration and input regime (lower is better) 38
Table A3. Held-out river test NRMSE under input-context ablation, weather inputs . . . 39
Table A4. Gain of the $\alpha = 0.75$ blend over message passing for each held-out outlet . . . 40
Table A5. Test MSE on the controlled tree task by query distance . . . 42
Table B1. Range measure on a 150-node path graph for untrained models . . . 46
Table C1. Pilot Jacobian slopes against effective resistance and test AP . . . 46
Table D1. Peptides-func model configurations and parameter counts . . . 47
Table D2. River model parameter counts and training time per epoch . . . 47
Table E1. Settings of the Higgs follow sweeps . . . 48

Introduction

Graph neural networks are the standard tool for learning on networked data such as social networks, molecules and transport systems. In each setting, the prediction for one node can depend on nodes several steps away. Whether a model can use that distant information depends on how the model moves information around, and the two main designs move it in opposite ways. A message-passing network moves information one edge per layer. A graph transformer lets every node attend to (look at) every other node in a single layer. Hybrid models combine both. There is still no principled way to decide in advance which design, or which mix, a given graph and task need.

Problem Statement

Current practice is trial and error: train a message-passing model, a graph transformer and a hybrid such as GPS (Rampášek et al., 2022), then keep the one with the best validation score. This costs several full training runs per task and does not explain why the winner won. The answer also does not transfer, because a score on one graph says little about the next.

The difficulty is that long-range learning can fail for two different reasons. *Over-squashing* happens when information from many distant nodes must squeeze through a few bridge nodes and gets compressed into fixed-size vectors (Alon & Yahav, 2021; Topping et al., 2022). Effective resistance, a measure of how hard it is for two nodes to communicate, bounds this effect (Di Giovanni et al., 2023; Black et al., 2023). *Attention dilution* is the opposite failure: when attention covers many nodes and does not concentrate, each relevant node gets only a small share of the focus (Xing et al., 2024). A model that reaches further avoids the first problem but invites the second. Benchmarks have not settled which matters when. Well-tuned message-passing models close most of the reported gap to transformers on the Long Range Graph Benchmark (Tönshoff et al., 2024), and some of its tasks turn out to need only short-range information (Bamberger et al., 2025).

Social graphs make the question sharper. They are large, they have hubs with thousands of followers, and almost every pair of users is joined by a short path. No existing long-range benchmark contains a social graph: LRGB has vision, chemistry and quantum-chemistry graphs (Dwivedi et al., 2022), and ECHO has synthetic long-diameter graphs and molecules (Miglior et al.,

2025). So we do not know whether long-range reasoning matters on social data, or how to tune a model for it.

Research Goals

The project asks one question: *can the structure of a graph and the range of a task predict how far a model's attention should reach?* We answer it with three components.

1. **Measurements on trained models.** Jacobian sensitivity against (excess) effective resistance for over-squashing; attention entropy for dilution; and the range measure of Bamberger et al. (2025) for how far a model's predictions actually reach.

2. **Controlled sweeps of one knob.** The main knob is the attention radius k: attention may only look at nodes within k hops. $k = 0$ is pure message passing and $k = \infty$ is full attention. We also sweep a fixed blend weight α between the two branches. Sweeps run on real targets and on planted targets whose range is known.

3. **A phase diagram.** Graph structure (excess resistance) on one axis, task range on the other, each cell coloured by the best setting of the knob.

We then ask the same reach question of graph foundation models, which are pretrained once and used on many graphs (Gutteridge et al., 2026; Eremeev et al., 2026), and we plan to release the planted-range social tasks as a benchmark.

We test four hypotheses. (H1) Message passing's Jacobian sensitivity falls faster with excess resistance than attention's. (H2) Attention entropy is high on large, well-connected social graphs, so full attention is exposed to dilution. (H3) Real social labels are mostly local, so the best radius on real targets is small. (H4) On planted targets the measured range of a well-trained model recovers the planted range, and the best radius grows with it.

Scope

The main experiments use four graphs from the Stanford Network Analysis Project (Leskovec & Krevl, 2014): the Twitch gamers network, a Twitch language network (German), the Reddit hyperlink network and the Twitter Higgs-boson follow network. Every model is built from one

shared layer, so that only the knob changes between variants. We train on a single GPU (Google Colab), which rules out large foundation models trained from scratch and wide hyperparameter searches. We study node-level prediction only.

This report replaces an earlier design. That earlier work used molecular graphs (Peptides-func), river networks and synthetic trees, plus a Reddit pilot. It showed that a hybrid helps only when a task needs information a local model cannot use, and that these graphs are near-trees, where resistance and hop distance cannot be told apart. It motivated the redesign and is summarised in Appendix .

Background

This section first explains the ideas behind the project in plain terms, then reviews the literature. Readers who know graph learning can skip to the Literature Review.

The Intuition Behind the Tasks

Predicting a Node from Its Neighbours

In *node prediction* we have one large graph. Each node (a user, a channel, a community) has some input features, and some nodes have a known label. The goal is to predict the label for the other nodes. Why should the graph help? Because connected nodes tend to be alike. Sociologists call this *homophily*: people link to people like themselves (McPherson et al., 2001). Connections also cause similarity, through *social influence*: if the people you follow start tweeting about a topic, you are more likely to join in. Either way, a node's neighbours carry evidence about the node. A model that ignores the graph and uses only a node's own features is our yardstick; we call it the MLP baseline (a plain neural network).

Message Passing: A Game of Telephone

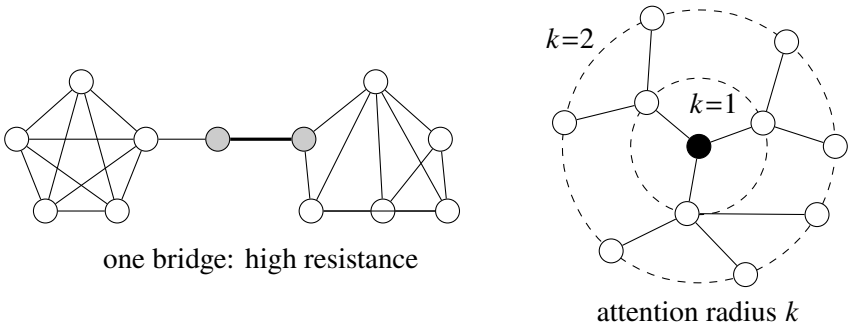
A message-passing network (Gilmer et al., 2017) updates each node by combining its own state with a summary of its neighbours' states. One layer reaches one hop. After L layers a node has heard, indirectly, from everything within L hops. Information from far away is relayed node by node, like a game of telephone: each relay mixes the message with everything else it has heard and passes on a fixed-size summary.

Over-Squashing: Many Voices Through a Narrow Bridge

Now suppose two dense groups of nodes are joined by a single edge (Figure 1, left). Everything one group says to the other must cross that bridge, and the bridge node can pass on only a fixed-size vector per layer. Many voices get squeezed into one channel and most of the detail is lost. This is *over-squashing* (Alon & Yahav, 2021; Topping et al., 2022).

Two measurements detect it. *Effective resistance* $R(u, v)$ treats every edge as a one-ohm resistor and asks for the electrical resistance between u and v. Many short parallel paths give low resistance. One long or narrow path gives high resistance, which signals a bottleneck. Theory shows that high resistance limits how much u can influence v in a message-passing network (Di Giovanni et al., 2023; Black et al., 2023). *Jacobian sensitivity* asks the question directly on a trained model: if we nudge node u's input, how much does node v's output move? If over-squashing bites, this sensitivity falls as resistance rises.

Figure 1. *Two pictures behind the project*



Note. Left: two dense groups joined by a single bridge (thick edge). Every message between the groups must cross it, so it is a bottleneck and the effective resistance between the groups is high. Right: with attention radius k, the black node may attend directly to every node inside the k-th dashed ring. The rings grow quickly with k: a larger radius adds shortcuts past bottlenecks but puts more nodes into each attention step.

Attention: A Direct Line, and Too Many Voices in the Room

A graph transformer (Vaswani et al., 2017; Dwivedi & Bresson, 2021) gives every node a direct line to every other node. In each layer, node v scores every node u for relevance, turns the scores

into weights that sum to one (a *softmax*), and takes a weighted average of their states. There is no bridge to cross, so over-squashing cannot happen in the same way. The cost is *dilution*. On a graph with 35,000 nodes, the weights must be shared among 35,000 candidates, most of them irrelevant. If attention does not learn to concentrate, the useful voice is drowned out by the crowd: too many voices in the room (Xing et al., 2024). We measure this with *attention entropy*, which is low when a node focuses on a few others and high when its attention is spread thin. Dividing by its maximum, $\log N$, gives a number between 0 (focused) and 1 (uniform).

Task Range: How Far Must a Node Listen?

Some tasks need only a node's close neighbours; others need nodes several hops away. Bamberger et al. (2025) turn this into a number. For a trained model, they measure how much each input node influences each output node (again through the Jacobian), and take the influence-weighted average distance. If a node's prediction depends mostly on nodes two hops away, its range is about two. A task's range is then a property we can measure, rather than a label we assume.

Why Social Graphs Are Interesting, and Hard

Social graphs are *small-world* networks (Watts & Strogatz, 1998). Most users are only a few hops from each other: on the Higgs follow graph the effective diameter (the distance within which 90% of pairs fall) is about 3.7 hops, and a user has about 30 follow links on average. Social graphs also have *hubs*, accounts with very many links. Both facts shape the experiments. First, k-hop neighbourhoods grow very fast, so by three hops a node can reach most of the graph. "Long range" therefore means two or three hops here, not twenty. Second, on large, well-connected graphs effective resistance collapses to a simple function of degree, $R(u, v) \approx 1/d_u + 1/d_v$, where d_u is the number of links of node u (von Luxburg et al., 2014). Raw resistance then mostly tells us whether the two endpoints are hubs, not whether a bottleneck lies between them. We therefore use *excess resistance*, the part of R left over after removing this degree term (Section).

Choosing a Clean Knob: Why α Was Flawed and Why the Radius k Is Better

Our first design placed every model on one dial. In each layer a hybrid computes a message-passing update and an attention update and mixes them as $(1 - \alpha) \cdot \mathrm{Norm}(\mathrm{MP}) + \alpha \cdot \mathrm{Norm}(\mathrm{Attention})$,

from $\alpha = 0$ (pure message passing) to $\alpha = 1$ (pure attention). The problem is in the normalisation layers. They come with learnable scale parameters, so during training the model can simply turn one branch up and the other down, undoing any α strictly between 0 and 1. The dial then sets only the starting point of training, not the final model. We fixed this by removing the learnable scales from the two branch norms, which makes α a true fixed weight.

The cleaner knob is the *attention radius* k (Figure 1, right): attention may only look at nodes within k hops. At $k = 0$ there is no attention and the model is pure message passing; at $k = \infty$ attention covers the whole graph. A larger k adds shortcuts past bottlenecks, which reduces over-squashing, but it also puts more nodes into each softmax, which increases dilution. The best k is exactly the trade-off the thesis is about. Unlike α, no learned weight can undo it, because nodes outside the radius are simply not visible. Restricting or reshaping attention by hop distance is not new (Gutteridge et al., 2023; Abboud et al., 2022; Ying et al., 2021); our contribution is to use the radius as a diagnostic knob alongside the measurements.

Planted Targets: A Known Answer

On real labels we cannot know the true range of the task, so we cannot tell whether the range measure is right or whether the best radius tracks it. *Planted* (semi-synthetic) targets solve this. We keep the real graph and the real features, add a few random input columns, and define the label from those columns so that it depends on nodes at exactly k hops and on nothing closer. The ground-truth range is then k by construction. This lets us check that the measured range of a trained model recovers k, and ask whether the best attention radius grows with k.

How the Experiments Fit Together

The work runs in phases S0 to S6 (Table 4 in the Methodology). In plain words: S0 asks what the social graphs look like and whether resistance means anything on them. S1 builds models that can run on graphs with tens of thousands of nodes. S2 asks whether reaching further helps on real social labels. S3 asks the same question when we know the right answer. S4 combines the results into the phase diagram. S5 asks whether pretrained graph foundation models reach far enough. S6 releases the planted tasks so others can test their models.

Literature Review

Over-squashing. Alon & Yahav (2021) showed that message-passing networks struggle when a node must summarise exponentially many distant nodes in a fixed-width vector, and named the effect over-squashing. Topping et al. (2022) tied it to negatively curved edges and proposed rewiring the graph. Di Giovanni et al. (2023) bounded the Jacobian sensitivity between two nodes in terms of width, depth and graph topology, and Black et al. (2023) showed that total effective resistance bounds over-squashing, which makes resistance a structural predictor that can be computed before training. Mishayev et al. (2025) argued that over-squashing can arise even between nearby nodes. These results describe what a model class *can* do; we measure what trained models *do*. For large graphs, von Luxburg et al. (2014) proved that resistance converges to $1/d_u + 1/d_v$, which is why we correct for degree.

Graph transformers and reach. Early graph transformers added Laplacian positional encodings to attention (Dwivedi & Bresson, 2021). Graphormer biased attention by shortest-path distance (Ying et al., 2021), and spectral attention made the encodings learnable (Kreuzer et al., 2021). GPS combined a message-passing layer and a global attention layer in every block (Rampášek et al., 2022). Exphormer (Shirzad et al., 2023) and the Performer (Choromanski et al., 2021) reduce the quadratic cost of attention, the latter with random features that give linear cost; we use the Performer for full-graph attention. Between the two extremes, DRew lets nodes connect to k-hop neighbours gradually as depth grows (Gutteridge et al., 2023), and Shortest Path Networks aggregate separately over each hop distance (Abboud et al., 2022). Our radius knob sits in this family.

Dilution. In language models, accuracy drops when relevant content sits in the middle of a long context (Liu et al., 2024). In graphs, Xing et al. (2024) described over-globalizing, where global attention puts weight on distant, irrelevant nodes and hurts accuracy, and Sun et al. (2025) linked a similar over-aggregation effect to the number of nodes attention covers. Gated and capacity-controlled global attention has been proposed as a remedy (Liu et al., 2026).

Measuring range, and benchmarks. The Long Range Graph Benchmark was designed

to need long-range interactions (Dwivedi et al., 2022), but Tönshoff et al. (2024) showed that tuned message-passing baselines close most of the gap, and Bamberger et al. (2025) found with their range measure that models on its Peptides tasks use mostly short-range information. ECHO (Miglior et al., 2025) builds tasks with controlled long-range dependencies, on synthetic graphs and molecules. Neither benchmark contains social graphs, and in social graphs "far" means a few hops through very large neighbourhoods, a different regime from long chains.

Graph foundation models. Gutteridge et al. (2026) showed that foundation models with a fixed message-passing backbone under-reach on tasks whose range differs from their training tasks. GraphPFN (Eremeev et al., 2026) is a node-level foundation model pretrained on synthetic graphs that uses attention to aggregate over neighbourhoods. NodeGround (Lee et al., 2026) found that tuned GNNs still beat foundation models overall on node classification, but did not analyse range. Testing an attention-based foundation model on planted-range social tasks extends both lines of work.

Social graph data. We use SNAP data (Leskovec & Krevl, 2014): the Reddit hyperlink network (Kumar et al., 2018), the Twitch networks (Rozemberczki et al., 2021; Rozemberczki & Sarkar, 2021), and the Twitter activity around the 2012 Higgs boson announcement (De Domenico et al., 2013).

Data

All main experiments use four public social graphs from SNAP (Leskovec & Krevl, 2014) (Table 1). We treat every graph as undirected for message passing and for distances. Each dataset has one real target, chosen so that it is not trivially predicted from a node's own features. On top of these real targets we add planted targets (Methodology).

Data Sources

Twitch gamers (Rozemberczki & Sarkar, 2021). Nodes are Twitch users and edges are mutual follows. Each user has account fields: views, account lifetime, creation and update dates, language, and flags for affiliate status and dead accounts. The target is the `mature` flag, whether a channel is

Table 1. *Social graphs and real targets*

Dataset	Graph	Target	Metric	Subgraphs (train/val/test)
twitch-gamers	168,114 users, 6.8M mutual-follow edges	`mature`: channel marked as explicit content (binary, about 47% positive)	AUROC	58/13/13 (2k nodes)
twitch-de	9,498 users, 153,138 edges	`mature` (binary)	AUROC	13/3/3 (500 nodes)
reddit-body	35,776 subreddits, 124,330 undirected links	`LIWC_Money`: share of money-related words in a community's posts (regression)	R^2	12/3/3 (2k nodes)
higgs-follow	35,593 users, 528,532 follow edges	`active_after`: user retweets, mentions or replies after the cutoff (binary, about 47% positive)	AUROC	12/3/3 (2k) or 49/11/11 (500)

Note. Subgraphs are METIS parts; in full-graph mode the same parts only decide which nodes are labelled in each split (Methodology). The Higgs graph has a second target, `activate_passive`, described below.

marked as explicit content. Views are log-scaled and language is one-hot encoded.

Twitch German network (Rozemberczki et al., 2021). A smaller, denser Twitch graph of German-language streamers. Inputs are the games a user plays, reduced to the 128 most common games, plus account fields (days active, partner status, log views). The target is again `mature`.

Reddit hyperlinks (Kumar et al., 2018). Nodes are subreddits (communities). An edge records a post in one subreddit that links to another; there are 286,561 such directed links, which give 124,330 undirected edges. Each post carries 86 numeric properties (text length, sentiment and LIWC word-category scores), and a subreddit's features are the averages over its outgoing posts. Subreddits with no outgoing posts have no features; they relay messages but are never scored. To pick a target that depends on the neighbourhood, we ranked candidate columns by how much a ridge regression improved when a node's own features were joined by the average features of its 1- to 4-hop neighbourhoods. `LIWC_Money` ranked first: R^2 rose from 0.108 with own features to 0.185 with four hops. Input columns that correlate above 0.9 with the target are removed so the model cannot copy it.

Twitter Higgs (De Domenico et al., 2013). This dataset records Twitter activity around the announcement of the Higgs boson discovery in July 2012: who follows whom, and timestamped

retweets, mentions and replies. The target `active_after` asks: will this user retweet, mention or reply to anyone after the cutoff, 4 July 2012 00:00 UTC (a few hours before the announcement)? It is a social-influence question: does the activity of the people you follow predict whether you join the conversation? The second target, `activate_passive`, is the same label scored only on the roughly 11% of users who had no activity before the cutoff. Their own history says nothing, so any signal must come from their neighbours.

Leakage controls for Higgs. A predictive model must not see the future. We enforce three rules. (1) The node set is users involved in at least one interaction *before* the cutoff; SNAP's full user set includes users who appear only because of later activity, which would leak the label through the node set itself. (2) The graph is the follow graph on that node set. The activity layers (retweet, reply, mention) are rebuilt from timestamped records before the cutoff only; SNAP's ready-made layers cover the whole week and are never used. (3) The inputs are follow in- and out-degree and, for each activity type, the pre-cutoff out-count, in-count and number of distinct partners. A check at build time asserts that every record used for inputs or layers is dated before the cutoff. One channel remains: SNAP does not say when the follow graph was collected, so follow links formed after the cutoff cannot be excluded.

Descriptive Analysis

Table 2 summarises the structure of the test subgraphs. Three properties shape the analysis.

Table 2. *Structure of the social graphs (mean over test subgraphs)*

Dataset	Mean degree	Effective diameter	Mean excess resistance	Spearman ρ
twitch-gamers	11.6	5.8	0.39	0.90
reddit-body	4.3	5.7	0.49	0.77
twitch-de	22.9	3.0	0.02	0.998
higgs-follow	16.7	3.7	0.09	0.98

Note. Effective diameter is the hop distance within which 90% of node pairs fall. Excess resistance is $R(u, v) - (1/d_u + 1/d_v)$ (Section). The last column is the Spearman rank correlation between resistance R and its degree-only limit $1/d_u + 1/d_v$; values near one mean resistance mostly measures degree.

First, the graphs are small-world. Effective diameters range from 3.0 to 5.8 hops inside

the subgraphs, so a few hops cover most nodes. Planted ranges larger than three or four hops are therefore not meaningful on these graphs.

Second, raw resistance mostly restates degree. Its rank correlation with $1/d_u + 1/d_v$ is 0.90 or higher on three of the four graphs and 0.77 on Reddit. Only the sparse Higgs retweet layer breaks the pattern (correlation 0.07), and that layer is too sparse to use. Excess resistance keeps the part that degree does not explain. It is largest on Reddit and Twitch gamers, which are the sparser graphs, and close to zero on the dense Twitch German graph. The graphs therefore cover a range of bottleneck levels, which the phase diagram needs.

Third, the Higgs activity layers before the cutoff are very sparse, with mean degree 1.4 for retweets and 0.16 for replies. We had planned to compare the same users under two different graph structures (follow versus reply), but these layers are too thin to train on, so all Higgs experiments use the follow graph.

Finally, partitioning cuts many edges. On Twitch gamers only 26% of edges fall inside the 2,000-node parts, and on Higgs-follow 32% of edges fall inside 500-node parts and 47% inside 2,000-node parts. In a small-world graph, any way of cutting it into pieces removes many links, which turned out to matter (Methodology, Training and Evaluation).

Methodology

The methodology follows three principles. Comparisons change one knob at a time, because every graph model is built from the same layer. Mechanisms are measured directly on trained models, not only inferred from accuracy. And each claim is checked against a case where we know the answer, either an analytic test case or a planted target. All code is a tested Python package, `gnn_mech`; earlier versions of the pipeline are described in Appendix .

Feature Engineering and Leakage Controls

The pipeline has four steps. (1) Download and parse each SNAP source into one cached graph. (2) Build node inputs and the target from the raw node table, with the Higgs leakage rules above. Skewed counts are log-scaled, and every input column is standardised with statistics from the training-labelled nodes only. Regression targets are standardised the same way. (3) Partition the

graph with METIS (Karypis & Kumar, 1998), a standard tool that cuts a graph into parts of roughly equal size with few edges between them. Whole parts are assigned to training, validation and test (70/15/15 of the parts), using a *split seed*. Because parts, not single nodes, are assigned, each split's labelled nodes form connected regions, which is closer to predicting for a new region of the network than random node sampling. (4) Optionally add a random-walk positional encoding (the probability that a random walk returns to its start after $1, \ldots, 16$ steps), which helps attention know where a node sits. It is used on subgraphs; full-graph runs do not use it.

The partitions decide which nodes are labelled. This is the key leakage control. A node's label is used for training only if its part is in the training split; validation and test labels are never used in the loss or for scaling. Nodes in other splits may still appear as unlabelled inputs in full-graph mode (below), as in standard transductive node classification (Kipf & Welling, 2017).

Planted Targets with a Known Range

Planted targets keep the real graph and real features (now nuisance inputs) and append a few random input columns. The label is an exact function of the planted columns on the graph, so its range is known. Let $S_k(v)$ be the *shell* of nodes exactly k hops from v, computed on the full graph. We use two families.

Shell mean (a sanity check). Each node gets one random value $z_u \sim \mathcal{N}(0, 1)$, and the label is the average of z over the k-hop shell, $y_v = \frac{1}{|S_k(v)|} \sum_{u \in S_k(v)} z_u$. This tests pure reach: a model must gather values from exactly k hops away. It has a known loophole on small-world graphs: when the shell is almost "everything at distance k or more", the label can be inferred from the inner nodes, because the total over the graph is fixed. We record how well this shortcut predicts the label.

Retrieve (the main test). Every node u carries a key, one of K discrete codes shown to the model as a random vector, and a value $w_u \sim \mathcal{N}(0, 1)$. Node v carries a query code. The query is chosen to match the key of a random node in v's k-hop shell, and no node closer than k hops carries that key. The label is the value of the matching shell node (the average, if several shell nodes share the key). The task is like asking a friend-of-a-friend-of-a-friend a question: the node must find the one distant node whose key matches its query and fetch its value. Nothing closer carries any

information about the answer, so the range is exactly k. Nodes farther than k hops may also carry the key; they are distractors that a model with unlimited reach must learn to ignore. Because shells grow very fast on social graphs, the number of key codes K is set separately for each k (from the 90th percentile of the k-hop neighbourhood sizes), so that the expected number of extra matches stays below one for most nodes.

Nodes without a valid target (an empty shell, or no eligible query) are left unlabelled. Targets are deterministic given the graph, family, k and a seed. For both families we compute the range of the target function itself from its exact Jacobian; it equals k by construction, and a unit test checks this. On Higgs-follow we use $k \in \{1, 2, 3, 4\}$. Shell sizes and eligible nodes per k on Higgs-follow are reported in the Findings.

Modeling Framework

The Hybrid Layer

Every graph model except the GCN baseline is a stack of the same layer. Given node states x and edges E, one layer computes

$$h_{\text{loc}} = \text{BN}_{\text{loc}}\big(\text{Local}(x, E)\big), \qquad h_{\text{att}} = \text{BN}_{\text{att}}\big(\text{Attn}(x)\big),$$
$$x' = x + \text{Dropout}\big((1-\alpha)\, h_{\text{loc}} + \alpha\, h_{\text{att}}\big), \qquad x'' = \text{BN}_{\text{ffn}}\big(x' + \text{FFN}(x')\big),$$

where BN is batch normalisation (Ioffe & Szegedy, 2015) and FFN is a small two-layer network applied to each node. When $\alpha = 0$ the attention branch is not built and the model is a pure message-passing network (MPNN); when $\alpha = 1$ the local branch is not built and the model is a pure transformer. A prediction head maps each node's final state to its output.

Local branch. We use a GCN-normalised message-passing step (Kipf & Welling, 2017): each node averages its neighbours' states, weighting each neighbour by $1/\sqrt{d_u d_v}$, then applies a linear map. Our first sweep used GINE (Hu et al., 2020), which sums neighbour messages; on social graphs the sum over a hub's hundreds of neighbours gave large, unstable messages, so we switched to the degree-normalised version.

Affine-free branch norms. With the default learnable scale and shift in $\mathrm{BN}_{\mathrm{loc}}$ and $\mathrm{BN}_{\mathrm{att}}$, training can rescale either branch and cancel α (Background). We set these two norms to have no learnable parameters, so α is a true fixed weight. The feed-forward norm keeps its parameters, since it does not affect the mix.

Attention Variants

Softmax attention is standard multi-head self-attention (Vaswani et al., 2017) over all nodes of a subgraph. Its cost grows with the square of the number of nodes, which is fine for 500-node subgraphs.

Performer attention (Choromanski et al., 2021) approximates softmax attention with random features, so its cost grows linearly with the number of nodes. We use it for full attention ($k = \infty$) on the full 35,593-node Higgs graph, with 64 random features per head. In evaluation the random features are fixed, so measurements are repeatable.

Radius attention is softmax attention in which node v may attend only to nodes within k hops. On small subgraphs it masks the dense attention matrix. On the full graph it works on a precomputed list of all node pairs within k hops, with a cap on the number of pairs to bound memory. On the Higgs follow graph there are 1.09 million pairs within one hop but 298.8 million within two, so only radius 0, 1 and ∞ fit on a single GPU there.

Model Variants and Baselines

Table 3 lists the variants. Both knobs live in the same layer. In the α sweep the attention covers the whole (sub)graph and α varies. In the radius sweep α is fixed at 0.5 and the radius k varies; $k = 0$ means no attention ($\alpha = 0$).

Unless stated otherwise, models have four layers, hidden width 64, four attention heads and dropout 0.1.

Training and Evaluation

Subgraph training. In our first design, each METIS part was a separate small graph: message passing and attention ran only inside the part, and a batch held eight parts. This is the usual way to

Table 3. *Model variants*

Variant	Description
MLP	No graph: each node is predicted from its own features only. The yardstick for any graph effect.
GCN	A plain graph convolutional network (Kipf & Welling, 2017) with residual connections, in the style of the tuned baselines of Tönshoff et al. (2024); same depth and width as the other models.
Hybrid, $\alpha \in \{0, .25, .5, .75, 1\}$	The layer above with full attention (softmax on subgraphs, Performer on the full graph). $\alpha = 0$ is the MPNN, $\alpha = 1$ the transformer.
Radius, $k \in \{0, 1, 2, \infty\}$	The layer above with $\alpha = 0.5$ and attention limited to k hops.

train on graphs too large for memory, and it keeps the splits fully separate. It turned out to be a poor fit for social graphs. On Higgs-follow only 32% of follow edges stay inside 500-node parts, and a logistic-regression check showed that the neighbour signal available with full-graph neighbours largely disappears with in-part neighbours (details in the Findings). Subgraph training throws away the edges the knobs are meant to probe, and planted k-hop targets would suffer even more, since their shells cross part boundaries.

Full-graph (transductive) training. Our main setup therefore keeps the whole graph. The METIS parts only decide which nodes are labelled for training, validation and test. Message passing and attention use all 35,593 Higgs nodes and all their edges in every split; only the loss and the scores are restricted to the split's labelled nodes. Every step processes the whole graph, so there is one optimiser step per epoch, and runs use several hundred epochs with a long early-stopping patience. Subgraph runs use up to 100 epochs with patience 20.

Optimisation. All models use AdamW (Loshchilov & Hutter, 2019) with learning rate 10^{-3}, five warm-up epochs then cosine decay, and gradient clipping at 1.0. Binary targets use cross-entropy loss and regression targets use squared error. We keep the checkpoint with the best validation score.

Metrics. For binary targets we report the area under the ROC curve (AUROC): the probability that a randomly chosen positive node is scored above a randomly chosen negative one. It is 0.5 for random guessing and 1 for a perfect ranking, and it does not depend on a threshold. We also record average precision. For regression targets (Reddit and the planted targets) we report R^2,

the share of the target's variance the model explains: 0 for predicting the mean, 1 for a perfect fit.

Seeds. Two kinds of randomness are separated. The *split seed* decides the METIS partition and which parts are training, validation and test. The *model seed* decides weight initialisation and dropout. On Higgs we use three split seeds and two model seeds per variant (six runs), and report the mean and standard deviation over runs; the first sweep used three model seeds on one split. Planted-target runs use one split seed and two model seeds. All choices of α and k are made on validation data.

Mechanistic Measurements

All measurements use the best checkpoint in evaluation mode. Each was first checked on cases with a known answer: resistance equals hop distance on a path; uniform attention gives normalised entropy one; a model that reads exactly the d-hop neighbour has range d.

Effective resistance and excess resistance. With L the graph Laplacian and L^+ its pseudo-inverse, $R(u, v) = (e_u - e_v)^\top L^+ (e_u - e_v)$, where e_u is the indicator vector of node u. It is computed exactly on graphs of up to 5,000 nodes and approximately above that. Because R tends to $1/d_u + 1/d_v$ on large graphs (von Luxburg et al., 2014), we report the excess resistance

$$\tilde{R}(u, v) = R(u, v) - \Big(\frac{1}{d_u} + \frac{1}{d_v}\Big),$$

the resistance left over after the degree term. High excess resistance means a bottleneck beyond what the endpoints' degrees explain.

Jacobian sensitivity. $J(u, v) = \|\partial h_v / \partial x_u\|$, the size of the change in node v's final state for a small change in node u's input. For sampled target nodes v, one backward pass gives the sensitivity to every source u. We relate $\log J$ to hop distance, raw resistance and excess resistance. A steeper fall with excess resistance means stronger over-squashing.

Attention entropy. For an attention layer and head, node i's attention weights a_{ij} over the N_i nodes it can see have entropy $-\sum_j a_{ij} \log a_{ij}$. Dividing by $\log N_i$ gives a value in $[0, 1]$: near 0 when focused, near 1 when spread evenly. It is computed per head, since averaging heads first

would inflate it. For softmax and radius attention the weights are exact. The Performer has no explicit attention matrix, but the row for a single query node can be computed exactly from its random features, so we compute entropy on sampled query nodes.

Range. Following Bamberger et al. (2025), the influence of source u on target v is $I_{vu} = \sum|\partial y_v/\partial x_u|$ (summed over input channels), and the range of v is the influence-weighted mean distance

$$\rho_v = \frac{\sum_u I_{vu}\, d(v,u)}{\sum_u I_{vu}},$$

with d the hop distance (or resistance). On planted targets we compare the measured range of each trained model with the planted k; a correct measure should rise with k.

Experimental Plan

Table 4 lists the phases, the question each answers and its status on 3 November 2026.

Table 4. *Experimental phases and status*

Phase	Question in plain words	Design	Status
S0	What do the social graphs look like, and does resistance mean anything on them?	Loaders, partitions, structure statistics, excess resistance	Done
S1	Can we run attention on graphs with tens of thousands of nodes?	Performer and radius attention, node-level models, MLP and GCN baselines	Done
S2	Does reaching further help on real social labels?	α and radius sweeps on real targets, on subgraphs and on the full graph, with all measurements	In progress
S3	Does the best reach follow the task's range when we know it?	Planted shell-mean and retrieve targets, k = 1–4, on Higgs	Being built
S4	Can structure and range predict the best setting?	Phase diagram: excess resistance × task range, coloured by the best knob setting	Planned
S5	Do graph foundation models reach far enough?	GraphPFN (and possibly GraphAny) on the planted tasks; measure their range	Planned
S6	Can others test their models on the same tasks?	Release generator code, node indices and seeds (no raw data)	Planned

A radius sweep on 500-node subgraphs has finished and is reported in the Findings as a

control. The running experiments are: (1) the full-graph α sweep and radius sweep on Higgs, for both targets, with three split seeds and two model seeds; and (2) the planted-target sweep on the full Higgs graph, with $k \in \{1, 2, 3\}$, both families and the variants MLP, GCN, MPNN ($\alpha = 0$), full attention ($\alpha = 1$), hybrid ($\alpha = 0.5$) and radius 1 (radius 2 does not fit in memory on this graph). *[Pending: Full-graph α and radius sweep results] [Pending: Planted-target sweep results and measured range versus planted k]*

Two scale limits apply. Small-world structure means that $k \geq 3$ covers most of the Higgs graph, so radius values beyond two are close to full attention, and planted ranges beyond four are not meaningful. And full-graph training needs the whole graph in GPU memory, which limits width and the number of attention pairs. The other datasets will move to full-graph or large-subgraph setups after Higgs.

Findings

This section reports what the social-graph work has shown so far. Table 5 lists every experiment, what it is for, and whether it has results. Three results are complete: the structure statistics of the graphs (S0), two α sweeps on the Higgs Twitter graph, and a diagnosis of why those sweeps showed no graph effect. The experiments that should answer the thesis question directly, full-graph sweeps and planted long-range targets, are running, and we describe how to read them when they finish. The earlier experiments on rivers, synthetic trees and Peptides-func are in Appendix A. They shaped this design but are not the main evidence.

Structure of the Social Graphs

Before training any model, we asked how much each graph is shaped by bottlenecks. A bottleneck is a place where many nodes can only reach each other through a few edges. Effective resistance measures this. Treat every edge as a one-ohm resistor; the resistance between two nodes is low when many short paths join them and high when they are joined through a narrow passage. High resistance is where over-squashing is expected (Black et al., 2023).

On large, well-connected graphs, raw resistance has a known weakness. Between two

Table 5. *Status of the social-graph experiments*

Experiment	Purpose	Status
S0 structure statistics	How bottlenecked each graph is, and whether resistance only measures degree	Complete
Higgs α sweep, first version	Does mixing in attention help a real social label?	Complete; α knob flawed
Higgs α sweep, affine-free	Same question with α as a true fixed weight	Complete
Subgraph diagnosis	Why no graph model beat the MLP	Complete
Full-graph α and radius sweeps	Same sweeps without cutting the graph	Running
Radius sweep on subgraphs	Attention radius k on 500-node parts	Done: $k = 1$ best by a hair, no gain over MLP
Planted targets (`shell_mean`(k), `retrieve`(k))	Tasks with a known range k	Running (being built)
Phase diagram (S4), GFM benchmark (S5), benchmark release (S6)	Answer the thesis question; test foundation models; release tasks	Planned

far-apart nodes it tends to $1/d_u + 1/d_v$, where d_u and d_v are the two nodes' degrees (von Luxburg et al., 2014). The intuition is that current flowing out of a node must first squeeze through that node's own edges. Once it reaches the well-connected middle of the graph, there are so many paths that the rest of the trip costs almost nothing. So raw resistance mostly tells you how many friends the two endpoints have, not whether a bottleneck lies between them. We therefore also report the *excess resistance* $R_{uv} - (1/d_u + 1/d_v)$, the part of the resistance that the endpoints' degrees do not explain.

Table 6 shows the results, averaged over the test subgraphs of each dataset. The effective diameter is the number of hops within which most pairs of nodes lie; a small value means the graph is "small-world" and every node is a few hops from almost every other.

Three things stand out. First, on three of the four main graphs, raw resistance is almost entirely explained by degree (Spearman correlation 0.90 or more). Any analysis that bins node pairs by raw resistance on these graphs would really be binning them by degree, which is why excess resistance is the structural axis we use from here on. Second, the Higgs follow graph and twitch-de have very little excess resistance (0.09 and 0.02). Their nodes are densely interconnected, so there

Table 6. *Structure of the social graphs (means over test subgraphs)*

Dataset	Graph	Mean deg.	Eff. diam.	Excess R	Spearman ρ
twitch-gamers	168,114 users, 6.8M edges	11.6	5.8	0.39	0.90
reddit-body	35,776 subreddits, 124,330 links	4.3	5.7	0.49	0.77
twitch-de	9,498 users, 153,138 edges	22.9	3.0	0.02	0.998
higgs-follow	35,593 users, 528,532 edges	16.7	3.7	0.09	0.98
higgs-retweet	Sparse retweet layer	–	–	–	0.07

Note. Excess R is the mean of $R_{uv} - (1/d_u + 1/d_v)$. Spearman ρ is the correlation between raw resistance and $1/d_u + 1/d_v$ over node pairs in the test subgraphs; a value near 1 means raw resistance is almost a function of degree. Before the 4 July 2012 cutoff the Higgs retweet and reply layers have mean degree 1.4 and 0.16, too sparse to use as designed.

are few true bottlenecks for attention to bypass. Twitch-gamers and Reddit have more (0.39 and 0.49), and longer diameters, so they are the better candidates for over-squashing. Third, the sparse Higgs retweet layer is the exception (correlation 0.07): with about one edge per node, it consists mostly of chains and small trees, where resistance behaves like path length rather than degree. It would be an interesting contrast, but it is too sparse for training before the cutoff.

Higgs: First α Sweep

The Higgs dataset follows Twitter users around the announcement of the Higgs boson discovery in July 2012 (De Domenico et al., 2013). Our main real task, `active_after`, asks whether a user will retweet, mention or reply after 4 July 2012, using only what was known before: the user's follow degree and their activity counts before the cutoff. About 47% of users are positive. The graph is the follow network among the 35,593 users who were active before the cutoff.

The intuition for why the graph could matter is social influence. If the people you follow start talking about the Higgs boson, you are more likely to join in. A model that looks at neighbours can pick that up; a model that only sees your own history cannot.

The first sweep cut the graph into 2,000-node subgraphs with METIS, a standard graph-partitioning algorithm that splits a graph into parts while cutting as few edges as it can (Karypis & Kumar, 1998). Whole subgraphs were assigned to training, validation and test. The hybrid used

a GINE message-passing branch and Performer attention, a linear-cost approximation of softmax attention (Choromanski et al., 2021). Its branch normalisations had learnable scales. Table 7 shows test AUROC (the probability that a random positive user is scored above a random negative one; 0.5 is chance).

Table 7. *Higgs* `active_after`*, first sweep: test AUROC (2,000-node subgraphs, 3 seeds)*

Model	Test AUROC
MLP (no graph)	**0.687** ± 0.000
GCN	0.684 ± 0.000
Hybrid, $\alpha = 0$ (message passing only)	0.652 ± 0.034
Hybrid, $\alpha = 0.25$	0.648 ± 0.003
Hybrid, $\alpha = 0.5$	0.669 ± 0.026
Hybrid, $\alpha = 0.75$	0.666 ± 0.019
Hybrid, $\alpha = 1$ (attention only)	0.677 ± 0.003

Note. Mean ± standard deviation over three seeds. Branch normalisations had learnable scales, so α was not a fixed weight (see text).

The MLP, which ignores the graph entirely, was the best model, and no hybrid came within its noise. Two problems showed up. GINE adds up its neighbours' messages, and on hub users with thousands of followers that sum becomes very large, which made $\alpha = 0$ unstable (±0.034). More importantly, we realised that α in this design is not a real knob. The layer computes $(1-\alpha)\,\mathrm{Norm}(\text{message passing}) + \alpha\,\mathrm{Norm}(\text{attention})$, and each normalisation has a learnable scale. Training can grow one branch's scale and shrink the other's, undoing whatever α we set. Any α strictly between 0 and 1 then fixes only the starting point of training, not the final model. We fixed both problems in the next sweep.

Higgs: Affine-Free α Sweep

The second sweep removed the learnable scales from the branch normalisations, so α is now a true fixed mixing weight. It replaced GINE with a degree-normalised GCN branch, which averages rather than sums neighbours and so stays stable on hubs. Because the subgraphs were smaller (500 nodes), we could use exact softmax attention instead of the Performer approximation. Smaller parts also gave about 11 test subgraphs instead of 3. We ran three different partition splits, each with two model seeds.

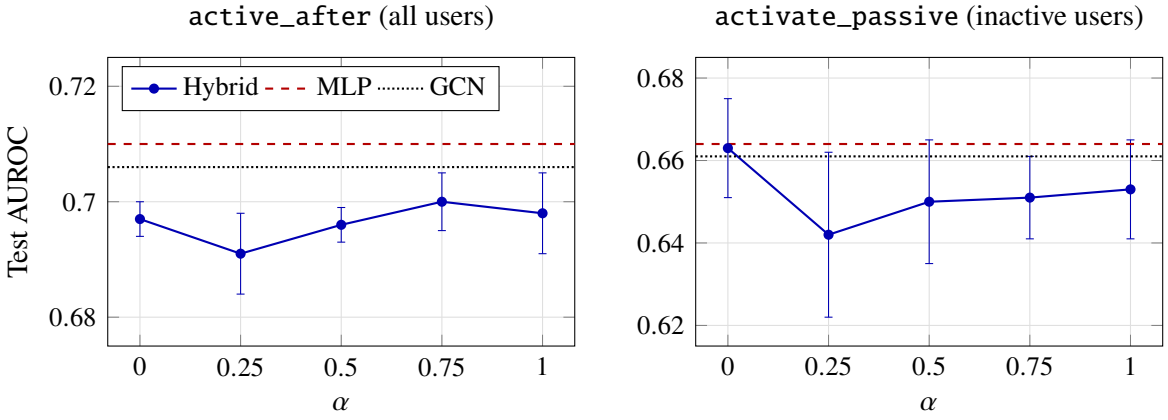
We added a second target, `activate_passive`. It has the same label but is scored only on the roughly 11% of users with no activity before the cutoff. For these users, their own history is empty: the only features left are follow counts. If the graph carries any signal, this is where it should show, because the model has nothing else to go on. Table 8 and Figure 2 show the results.

Table 8. *Higgs affine-free α sweep: test AUROC (500-node subgraphs)*

Model	`active_after`	`activate_passive`
MLP (no graph)	**0.710** ± 0.012	**0.664** ± 0.009
GCN	0.706 ± 0.010	0.661 ± 0.013
Hybrid, $\alpha = 0$	0.697 ± 0.003	0.663 ± 0.012
Hybrid, $\alpha = 0.25$	0.691 ± 0.007	0.642 ± 0.020
Hybrid, $\alpha = 0.5$	0.696 ± 0.003	0.650 ± 0.015
Hybrid, $\alpha = 0.75$	0.700 ± 0.005	0.651 ± 0.010
Hybrid, $\alpha = 1$	0.698 ± 0.007	0.653 ± 0.012

Note. Mean ± standard deviation over 3 split seeds × 2 model seeds. For `active_after` at $\alpha = 0$, 0.25 and 0.5, $n = 4$ of 6 because one batch of runs failed on a storage error. GCN local branch, exact softmax attention, branch normalisations without learnable scale.

Figure 2. *Test AUROC against α on Higgs, affine-free sweep, with MLP and GCN reference lines*



Note. Points are means and bars one standard deviation across split and model seeds (Table 8). The two panels have different vertical scales.

Again no graph model beat the MLP, on either target. Three readings follow.

An MLP that beats every graph model means the label is mostly predictable from a user's own features. The MLP sees exactly the same inputs as the graph models, minus the neighbours. If the neighbours carried useful information that the graph models could extract, at least one of

them should do better. When none does, either the neighbour signal is weak, or the models were not given access to it. The next subsection shows that the second explanation applies here.

The passive users are where the graph should matter, and it still did not help. For `active_after`, past activity is a strong predictor (people who tweeted before tend to tweet again), so a small neighbour effect is easy to miss. For `activate_passive` that shortcut is gone. That message passing at $\alpha = 0$ only matched the MLP here (0.663 against 0.664), and adding attention made things slightly worse (0.642 to 0.653), was the first sign that something in the setup was hiding the neighbours.

A flat α curve tells you less than it seems to. On `active_after`, all five hybrids lie between 0.691 and 0.700, with seed-to-seed spreads of similar size. This does not show that attention and message passing are equivalent, or that over-squashing and dilution are absent. It shows that, for this task and this setup, neither mechanism delivered information the model could use beyond a user's own features. A flat curve is what you would expect whenever the task does not depend on distant nodes, whatever the graph's structure. Only a task that needs distant information can tell the two mechanisms apart, which is why we built planted targets with a known range.

Why the Graph Did Not Help: Subgraph Partitioning

To check whether there was any neighbour signal to find, we fitted a plain logistic regression three ways: on a user's own features; on own features plus the averages of their one- and two-hop neighbours' features in the full graph; and on the same averages computed only from neighbours inside the user's METIS part. The last version sees exactly the neighbours a subgraph-trained model sees. Table 9 shows test AUROC.

There is a real neighbour signal, but it is modest. With the full graph, neighbours add 0.018 AUROC over own features for all users and 0.028 for passive users. As expected, the gain is larger where a user's own history is empty. Partitioning destroys most of it. With 500-node parts, only 32% of follow edges stay inside a part, and the neighbour gain for all users falls from 0.018 to 0.001. For passive users it falls from 0.028 to 0.013, and with 2,000-node parts it falls from 0.013

Table 9. *Neighbour signal with and without cutting the Higgs follow graph (logistic regression, test AUROC)*

Part size	Edges kept	Users	Own	+ nbrs (full graph)	+ nbrs (in part)
500	32%	All	0.672	**0.690**	0.673
500	32%	Passive	0.677	**0.705**	0.690
2,000	47%	All	0.694	**0.707**	0.700

Note. "Own" uses a user's own features only; "+ nbrs" adds the means of one- and two-hop neighbours' inputs. "In part" uses only neighbours inside the same METIS part. "Edges kept" is the share of follow edges with both ends in the same part.

to 0.006.

The intuition is simple. Message passing can only use the edges it is given. A part keeps all the nodes in it but drops every edge that crosses into another part. In a road network or a molecule, a good partition cuts few edges, because the graph has natural seams. Social graphs are small-world: a typical user is three or four hops from almost everyone, and friendships cross every boundary you could draw. Even the best partition must cut most edges, so each user loses most of their neighbours, and with them the signal. This explains why the MLP was hard to beat in both sweeps. It is a central and honest finding of this project: *on small-world social graphs, subgraph training throws away the edges the models need*. For any long-range target this is worse still, because a k-hop neighbourhood almost always crosses part boundaries. All later experiments therefore train on the full graph.

Full-Graph α and Radius Sweeps

Design. The full-graph (transductive) setup keeps the whole 35,593-node Higgs follow graph in every forward pass. The METIS parts now only decide which users' labels are used for training, validation and testing. Message passing and attention run over all nodes and all 528,532 edges. Every model sees every user's features, but only training labels are used to fit the model. Training uses the whole graph at once, so there is one optimiser step per epoch. Two sweeps run on both targets, with three split seeds and two model seeds, a GCN local branch and affine-free normalisations. The first sweep varies $\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$ with Performer attention, because exact softmax attention over 35,593 nodes would not fit in memory. The second sweep varies the *attention radius* $k \in \{0, 1, \infty\}$: attention may only look at nodes within k hops ($k = 0$ is pure

message passing, $k = \infty$ is full attention via Performer). Radius 2 is left out because it does not fit on the graph: there are 1.09 million attention pairs within one hop but 298.8 million within two, which would need tens of gigabytes per layer. MLP and GCN baselines run with each sweep.

The radius is the new main knob because, unlike α, training cannot undo it. A larger k gives each node direct shortcuts past bottlenecks, which reduces over-squashing, but also puts more nodes into each softmax, which increases dilution. The best k is the balance point between the two failures, which is exactly what the thesis is about. Related architectures use k-hop connections or distance information to improve accuracy (Gutteridge et al., 2023; Abboud et al., 2022; Ying et al., 2021); we use the radius as a controlled diagnostic, paired with the measurements.

How to read the outcomes. If graph models now beat the MLP, especially on `activate_passive` and by roughly the logistic-regression gap of 0.02 to 0.03 AUROC, subgraphing was the cause of the earlier null results. If $k = 1$ or $k = 2$ beats both $k = 0$ and $k = \infty$, a little attention beyond the immediate neighbours helps, but attending to everyone dilutes the signal. If the curves are still flat and the MLP still wins, the Higgs label is local: what matters is a user's own history and perhaps their immediate neighbours. In that case the real target cannot separate the mechanisms, and the planted targets below become the main test.

Results. [Pending: Full-graph affine-free α sweep on Higgs, both targets: test AUROC table and α curve, with MLP and GCN references.] [Pending: Full-graph attention-radius sweep, $k \in \{0, 1, 2, \infty\}$, both targets: test AUROC, number of attention pairs at $k = 1$ and $k = 2$, and attention entropy per k.]

Planted Long-Range Targets

Why plant a target? Real social labels do not come with a known range, so a flat result cannot tell us whether the mechanisms did not matter or the task did not need them. A planted target fixes this. We keep the real Higgs graph and the real user features, which become nuisance inputs, and add a few random input channels. The label is an exact function of those channels at a chosen distance k. The true range of the task is then k by construction. We checked this in unit tests with the range measure of Bamberger et al. (2025) applied to the target function itself.

Two families. `shell_mean`(k) is the sanity check. Every user gets a random number, and the label is the average of those numbers over the users exactly k hops away. A message-passing model with at least k layers can in principle compute it. `retrieve`(k) is the main test. Every user carries a random key and a random value. A user's query matches the key of one user exactly k hops away, and no closer user carries that key. The label is the matched user's value. This is a lookup, not an average. Message passing has to carry many key–value pairs through every intermediate user and keep them separate, which is where over-squashing should bite. Attention can match a query to a key directly, but users farther than k hops may share the key, so attention that looks too far collects distractors and dilutes the answer. The number of distinct keys is chosen separately for each k so that each query has about one expected match.

Design. Both families run for $k \in \{1, 2, 3\}$ on the full graph, with models MLP, GCN, hybrid at $\alpha \in \{0, 0.5, 1\}$, and attention radius 1, using two model seeds. We dropped $k = 4$, where the four-hop neighbourhood covers 97% of the graph, and radius 2, which does not fit in memory on this graph. For each trained model we measure its range, the influence-weighted average distance of the inputs it actually uses (Bamberger et al., 2025), and compare it with the planted k.

What would support the thesis. On `retrieve`(k), the best attention radius should grow with k. On Higgs the radius knob can only take the values 0, 1 and ∞, so the test is narrower: radius 1 should win at $k = 1$, full attention should win at $k \geq 2$, pure message passing should degrade as k grows, and full attention should pay for its distractors at $k = 1$. A graph with slower-growing neighbourhoods, such as Reddit, is needed to test intermediate radii. The measured range of the best model at each k should also track k; the planned gate is a Spearman correlation above 0.8. Together these would show that task range predicts where a model should sit between local and global, and that the range measure can see it. Results against the thesis would include: one setting winning at every k; message passing solving `retrieve` at every k without loss; or a measured range that ignores k.

A caveat built into the graph. The Higgs follow graph has an effective diameter of about 3.7 hops and a mean degree of about 30 in the full graph (16.7 inside test subgraphs, because

partitioning cuts edges), so neighbourhoods grow very fast. The median user has about 30 users at one hop and thousands at two, and the three-hop neighbourhood covers most of the graph. For $k \geq 3$ there is little difference between "within k hops" and "everywhere", so a radius sweep can only resolve k up to about 2. For `shell_mean` at $k \geq 3$ there is a further leak: the k-shell is roughly the whole graph minus the inner ball, so a model can predict the label from nearby nodes. We report this leak alongside the results.

Results. Table 10 shows how fast neighbourhoods grow on the Higgs follow graph. Almost every user is eligible at every k, but the two-hop ball already holds about a quarter of the graph and the three-hop ball most of it. The table also shows a shortcut that full attention can take on `retrieve`: averaging the value of every user whose key matches the query, at any distance, already predicts the label well at $k \geq 2$ (R^2 of 0.53 at $k = 2$ and 0.90 at $k = 3$), because at that scale the matching key is nearly unique in the whole graph. So at large k, `retrieve` tests whether a model can reach far, but puts little pressure on dilution.

Table 10. *Neighbourhood growth and planted-target checks on the Higgs follow graph*

k	Eligible	Median shell	Ball share	Key matches	Key-match R^2
1	98.1%	10	0%	1.51	0.00
2	97.7%	8,322	23%	1.48	0.53
3	97.6%	19,537	84%	1.56	0.90
4	97.6%	4,434	97%	1.20	0.98

Note. Eligible: users with a defined target. Median shell: median number of users at exactly k hops. Ball share: median share of the graph within k hops. Key matches: mean number of users at hop k matching the query. Key-match R^2: how well the label of `retrieve`(k) is predicted by averaging the value of every user with the query key, ignoring distance. The analytic range of both target families equals k exactly at every k. $k = 4$ is excluded from the sweep.

[Pending: Test error of each model on `shell_mean`(k) *and* `retrieve`(k)*, k = 1–3.]*
[Pending: Measured range of each trained model against planted k, with the Spearman correlation for the best model per k.]

Attention Radius on Subgraphs

Design. This sweep is the radius counterpart of the affine-free sweep and uses the same 500-node subgraphs, targets, seeds and baselines. It fixes $\alpha = 0.5$ and varies the attention radius

$k \in \{0, 1, 2, 3, 4, \infty\}$ with exact softmax attention inside each subgraph. It was launched before the partition diagnosis, and it shares that problem: every radius sees only the 32% of edges inside the parts. We keep it as a control. If its results are flat while the full-graph radius sweep is not, that confirms that cutting the graph, not the radius knob, hid the effect. If both are flat, the Higgs labels are local.

Results. All 96 runs finished (Table 11). The pattern is faint but points the same way on both targets: attention limited to one hop is the best radius, and letting attention look further costs a little. On `active_after`, test AUROC falls from 0.705 at $k = 1$ to 0.698 at $k = 4$ and 0.700 at $k = \infty$. On `activate_passive` it falls from 0.663 at $k = 1$ to about 0.653 for every $k \geq 2$. This is the direction that attention dilution predicts: each extra hop adds many more users to every softmax, and most of them carry nothing useful, so attention spread over them gets noisier.

Three things keep us from reading much into it. First, every gap is smaller than one standard deviation across split and model seeds. Second, no radius beats the baselines: the MLP is best on `active_after` and GCN on `activate_passive`. Third, the sweep ran on the same cut-up subgraphs as the affine-free sweep, so every model saw only about a third of the edges. Here $k = \infty$ means "the whole 500-node part", not the whole graph. The sweep is therefore a control. It shows that the radius knob runs and behaves sensibly, but the full-graph sweep is the real test.

Table 11. *Attention radius on 500-node Higgs subgraphs: test AUROC*

Variant	`active_after`	`activate_passive`
MLP (no graph)	**0.710** ± 0.012	0.664 ± 0.009
GCN	0.706 ± 0.010	**0.666** ± 0.011
$k = 0$ (message passing only)	0.704 ± 0.011	0.660 ± 0.011
$k = 1$	0.705 ± 0.009	0.663 ± 0.006
$k = 2$	0.703 ± 0.008	0.654 ± 0.008
$k = 3$	0.702 ± 0.011	0.653 ± 0.010
$k = 4$	0.698 ± 0.006	0.653 ± 0.014
$k = \infty$ (whole subgraph)	0.700 ± 0.006	0.654 ± 0.011

Note. Mean ± standard deviation over 3 split seeds × 2 model seeds (6 runs each). Hybrid variants use $\alpha = 0.5$, a GCN local branch, affine-free branch norms and exact softmax attention masked to k hops; $k = 0$ has no attention branch.

Discussion

What the Results So Far Mean

The completed social-graph results are negative for the graph models, and they are informative. On Higgs, a model that ignores the graph matched or beat every graph model in two sweeps. The diagnosis explains why: the real neighbour signal is modest (0.018 to 0.028 AUROC in a linear model), and cutting the graph into parts removed most of it. Neither sweep was a fair test of message passing against attention, because both mechanisms were starved of the edges they work on. The practical lesson for anyone training GNNs on social data is to check how many edges survive a partition before trusting a subgraph experiment. On small-world graphs the answer can be a third.

The work also changed how we probe the mechanisms. The α knob as first built did not do what its name said, because learnable normalisation scales could undo it. Removing them makes α honest, but the attention radius is a better knob. Learned weights cannot undo it, and it maps directly onto the trade-off between reaching past bottlenecks and spreading attention thin. The structure statistics changed the structural axis as well. Raw effective resistance on these graphs is mostly a degree statistic, so the phase diagram uses excess resistance.

Limitations

Small-world graphs limit radius resolution. With effective diameters of 3.0 to 5.8 hops and neighbourhoods that reach most of the graph within three hops on Higgs, radii above 2 are close to full attention. The radius knob can separate "immediate neighbours", "neighbours of neighbours" and "everyone", but not much more. Reddit and twitch-gamers, with longer diameters and more excess resistance, give more room.

Resistance is close to degree. Spearman correlations of 0.77 to 0.998 between raw resistance and $1/d_u + 1/d_v$ mean that raw resistance adds little beyond degree. Excess resistance removes the degree part, but on Higgs and twitch-de it is small (0.09 and 0.02), so these graphs have little bottleneck structure to vary. The structural axis of the phase diagram will be narrow on them.

Few independent test units. The independent unit for uncertainty is a test subgraph or

partition split, not a user, because users in the same part are correlated. The sweeps had 3 to about 11 test subgraphs and three partition splits. The standard deviations in Tables 7 and 8 are across a handful of runs, and some cells have $n = 4$, so differences of 0.01 AUROC are not resolved.

Real social labels are mostly local. The Higgs label is predicted mainly by a user's own history, and the neighbour gain in a linear model is at most about 0.03 AUROC. Real labels like this cannot, on their own, show whether a model reaches far enough, which is why the planted targets carry the main test. The planted targets in turn are synthetic signals on a real graph. They control the task range but not how realistic the task is.

Undated follow edges. The Higgs follow graph has no timestamps. Some follow edges may have been created after the cutoff, possibly because of the Higgs activity itself. The node features use only pre-cutoff activity, but the graph cannot be fully restricted to the past, which may inflate the neighbour signal slightly.

Performer approximates softmax. Full attention on the whole graph uses Performer, a random-feature approximation of softmax attention with linear cost. Its attention weights are only approximately those of softmax, so its measured entropy and dilution may differ from those of exact attention. Radius-limited attention uses exact softmax over the allowed pairs, so comparisons between finite k and $k = \infty$ mix the radius with the attention type. We report this wherever the two are compared.

How the Remaining Phases Answer the Thesis Question

The thesis question is whether graph structure and task range together predict where a model should sit between local and global. The remaining phases answer it in three steps. The phase diagram (S4) takes the planted tasks on each graph and plots the best setting, the attention radius or α chosen on validation data, against excess resistance between a node and its planted sources on one axis and task range on the other, with uncertainty from resampling test subgraphs. If the thesis holds, the diagram shows regions: message passing where the range is short and bottlenecks are mild, wider attention where the range is long or bottlenecks are severe, and an overlay of attention entropy marking where dilution begins.

The graph foundation model benchmark (S5) applies the same tasks to models trained once and then used on new graphs. Gutteridge et al. (2026) showed that foundation models with a fixed message-passing backbone under-reach on tasks whose range differs from what they were trained on. We will test whether this also holds for an attention-based node-level foundation model, GraphPFN (Eremeev et al., 2026), and possibly GraphAny, on social graphs, by measuring each model's range on `retrieve`(k) and comparing it with the required k. A recent node-classification benchmark found that tuned GNNs still beat foundation models overall but did not analyse range (Lee et al., 2026). Finally, the benchmark release (S6) publishes the planted-range social tasks as generator code, node indices and seeds. No existing long-range benchmark contains social graphs: LRGB covers vision, chemistry and quantum chemistry (Dwivedi et al., 2022), and ECHO covers synthetic long-diameter graphs and molecules (Miglior et al., 2025).

Conclusion

We set out to show that graph structure and task range predict whether a graph model should use local message passing, global attention, or a mix. On large social graphs we have not yet shown this. What we have shown is what stands in the way. Raw effective resistance on these graphs mostly measures degree, so bottlenecks must be measured by excess resistance. The original α knob could be undone by training, so we replaced it with a fixed weight and an attention radius. Most importantly, cutting small-world graphs into subgraphs throws away most of the edges, and with them the neighbour signal. This is why a plain MLP matched every graph model on the Higgs Twitter task.

The full-graph sweeps and the planted long-range targets are running. They are the first experiments in this project in which the task range is known and the graph is intact. If the best attention radius grows with the planted range and the measured range of the best model tracks it, the phase diagram and the foundation model benchmark follow directly. If not, the same measurements will show which assumption failed. For practitioners, the advice that holds already is: before comparing graph architectures, check that the graph signal survives your training setup and that the task actually needs distant information.

References

Alon, U., & Yahav, E. (2021). On the bottleneck of graph neural networks and its practical implications. In *International Conference on Learning Representations*. https://arxiv.org/abs/2006.05205

Bamberger, J., Gutteridge, B., Le Roux, S., Bronstein, M. M., & Dong, X. (2025). On measuring long-range interactions in graph neural networks. In *Proceedings of the 42nd International Conference on Machine Learning* (Vol. 267, pp. 2770–2789). PMLR.

Black, M., Wan, Z., Nayyeri, A., & Wang, Y. (2023). Understanding oversquashing in GNNs through the lens of effective resistance. In *Proceedings of the 40th International Conference on Machine Learning* (Vol. 202, pp. 2528–2547). PMLR.

Di Giovanni, F., Giusti, L., Barbero, F., Luise, G., Liò, P., & Bronstein, M. M. (2023). On over-squashing in message passing neural networks: The impact of width, depth, and topology. In *Proceedings of the 40th International Conference on Machine Learning* (Vol. 202, pp. 7865–7885). PMLR.

Dwivedi, V. P., & Bresson, X. (2021). *A generalization of transformer networks to graphs* [Paper presentation]. AAAI 2021 Workshop on Deep Learning on Graphs: Methods and Applications (DLG-AAAI 2021). https://arxiv.org/abs/2012.09699

Dwivedi, V. P., Rampášek, L., Galkin, M., Parviz, A., Wolf, G., Luu, A. T., & Beaini, D. (2022). Long range graph benchmark. In *Advances in Neural Information Processing Systems* (Vol. 35, pp. 22326–22340).

Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., & Dahl, G. E. (2017). Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning* (Vol. 70, pp. 1263–1272). PMLR.

Hamilton, W. L., Ying, R., & Leskovec, J. (2017). Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems* (Vol. 30).

Hu, W., Liu, B., Gomes, J., Zitnik, M., Liang, P., Pande, V., & Leskovec, J. (2020). Strategies for pre-training graph neural networks. In *International Conference on Learning Representations*.

Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*.

Klingler, C., Schulz, K., & Herrnegger, M. (2021). LamaH-CE: LArge-SaMple DAta for hydrology and environmental sciences for Central Europe. *Earth System Science Data, 13*, 4529–4565. https://doi.org/10.5194/essd-13-4529-2021

Kratzert, F., Klotz, D., Shalev, G., Klambauer, G., Hochreiter, S., & Nearing, G. (2019). Towards learning universal, regional, and local hydrological behaviors via machine learning applied to large-sample datasets. *Hydrology and Earth System Sciences, 23*(12), 5089–5110.

https://doi.org/10.5194/hess-23-5089-2019

Kreuzer, D., Beaini, D., Hamilton, W. L., Létourneau, V., & Tossou, P. (2021). Rethinking graph transformers with spectral attention. In *Advances in Neural Information Processing Systems* (Vol. 34, pp. 21618–21629).

Kumar, S., Hamilton, W. L., Leskovec, J., & Jurafsky, D. (2018). Community interaction and conflict on the web. In *Proceedings of the 2018 World Wide Web Conference* (pp. 933–943). https://doi.org/10.1145/3178876.3186141

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., & Liang, P. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics, 12*, 157–173. https://doi.org/10.1162/tacl_a_00638

Liu, Y., Guo, D., Zheng, T., Yiu, S. M., Ning, L., & Wu, J. (2026). *Capacity-controlled global attention for graph transformers* (arXiv:2604.17324). arXiv. https://doi.org/10.48550/arXiv.2604.17324

Loshchilov, I., & Hutter, F. (2019). Decoupled weight decay regularization. In *International Conference on Learning Representations*.

Ma, L., Lin, C., Lim, D., Romero-Soriano, A., Dokania, P. K., Coates, M., Torr, P., & Lim, S.-N. (2023). Graph inductive biases in transformers without message passing. In *Proceedings of the 40th International Conference on Machine Learning* (Vol. 202, pp. 23321–23337). PMLR.

Miglior, L., Tolloso, M., Gravina, A., & Bacciu, D. (2025). *Can you hear me now? A benchmark for long-range graph propagation* (arXiv:2512.17762). arXiv. https://doi.org/10.48550/arXiv.2512.17762

Mishayev, Y., Sverdlov, Y., Amir, T., & Dym, N. (2025). *Short-range oversquashing* (arXiv:2511.20406). arXiv. https://doi.org/10.48550/arXiv.2511.20406

Nash, J. E., & Sutcliffe, J. V. (1970). River flow forecasting through conceptual models part I—A discussion of principles. *Journal of Hydrology, 10*(3), 282–290. https://doi.org/10.1016/0022-1694(70)90255-6

Rampášek, L., Galkin, M., Dwivedi, V. P., Luu, A. T., Wolf, G., & Beaini, D. (2022). Recipe for a general, powerful, scalable graph transformer. In *Advances in Neural Information Processing Systems* (Vol. 35, pp. 14501–14515).

Shirzad, H., Velingker, A., Venkatachalam, B., Sutherland, D. J., & Sinop, A. K. (2023). Exphormer: Sparse transformers for graphs. In *Proceedings of the 40th International Conference on Machine Learning* (Vol. 202, pp. 31613–31632). PMLR.

Sun, J., Chang, W., Yang, C., Huang, Q., & Wang, S. (2025). Relieving the over-aggregating effect in graph transformers. In *Advances in Neural Information Processing Systems* (Vol. 38, pp. 6673–6699).

Tönshoff, J., Ritzert, M., Rosenbluth, E., & Grohe, M. (2024). Where did the gap go? Reassessing the long-range graph benchmark. *Transactions on Machine Learning Research*. https://openreview.net/forum?id=Nm0WX86sKv

Topping, J., Di Giovanni, F., Chamberlain, B. P., Dong, X., & Bronstein, M. M. (2022). Understanding over-squashing and bottlenecks on graphs via curvature. In *International Conference on Learning Representations*.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems* (Vol. 30).

Xing, Y., Wang, X., Li, Y., Huang, H., & Shi, C. (2024). Less is more: On the over-globalizing problem in graph transformers. In *Proceedings of the 41st International Conference on Machine Learning* (Vol. 235, pp. 54656–54672). PMLR.

Xu, K., Hu, W., Leskovec, J., & Jegelka, S. (2019). How powerful are graph neural networks? In *International Conference on Learning Representations*.

Ying, C., Cai, T., Luo, S., Zheng, S., Ke, G., He, D., Shen, Y., & Liu, T.-Y. (2021). Do transformers really perform badly for graph representation? In *Advances in Neural Information Processing Systems* (Vol. 34, pp. 28877–28888).

Abboud, R., Dimitrov, R., & Ceylan, İ. İ. (2022). Shortest path networks for graph property prediction. In *Proceedings of the First Learning on Graphs Conference* (Vol. 198). PMLR. https://arxiv.org/abs/2206.01003

Choromanski, K., Likhosherstov, V., Dohan, D., Song, X., Gane, A., Sarlos, T., Hawkins, P., Davis, J., Mohiuddin, A., Kaiser, Ł., Belanger, D., Colwell, L., & Weller, A. (2021). Rethinking attention with Performers. In *International Conference on Learning Representations*. https://arxiv.org/abs/2009.14794

De Domenico, M., Lima, A., Mougel, P., & Musolesi, M. (2013). The anatomy of a scientific rumor. *Scientific Reports, 3*, 2980. https://doi.org/10.1038/srep02980

Eremeev, D., Platonov, O., Bazhenov, G., Babenko, A., & Prokhorenkova, L. (2026). GraphPFN: A prior-data fitted graph foundation model. In *Proceedings of the 43rd International Conference on Machine Learning*. https://arxiv.org/abs/2509.21489

Gutteridge, B., Dong, X., Bronstein, M. M., & Di Giovanni, F. (2023). DRew: Dynamically rewired message passing with delay. In *Proceedings of the 40th International Conference on Machine Learning* (Vol. 202). PMLR. https://arxiv.org/abs/2305.08018

Gutteridge, B., Bronstein, M. M., & Dong, X. (2026). *Can graph foundation models generalize over architecture?* (arXiv:2603.22984). arXiv. https://doi.org/10.48550/arXiv.2603.22984

Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on*

Machine Learning (Vol. 37, pp. 448–456). PMLR.

Karypis, G., & Kumar, V. (1998). A fast and high quality multilevel scheme for partitioning irregular graphs. *SIAM Journal on Scientific Computing, 20*(1), 359–392. https://doi.org/10.1137/S1064827595287997

Lee, J., Lee, D., Jeong, M., & Yoo, J. (2026). *NodeGround: A node classification benchmark in the graph foundation model era* (arXiv:2609.39673). arXiv. https://doi.org/10.48550/arXiv.2609.39673

Leskovec, J., & Krevl, A. (2014). *SNAP Datasets: Stanford large network dataset collection* [Data set]. http://snap.stanford.edu/data

McPherson, M., Smith-Lovin, L., & Cook, J. M. (2001). Birds of a feather: Homophily in social networks. *Annual Review of Sociology, 27*, 415–444. https://doi.org/10.1146/annurev.soc.27.1.415

Rozemberczki, B., Allen, C., & Sarkar, R. (2021). Multi-scale attributed node embedding. *Journal of Complex Networks, 9*(2), cnab014. https://doi.org/10.1093/comnet/cnab014

Rozemberczki, B., & Sarkar, R. (2021). *Twitch gamers: A dataset for evaluating proximity preserving and structural role-based node embeddings* (arXiv:2101.03091). arXiv. https://doi.org/10.48550/arXiv.2101.03091

von Luxburg, U., Radl, A., & Hein, M. (2014). Hitting and commute times in large random neighborhood graphs. *Journal of Machine Learning Research, 15*, 1751–1798.

Watts, D. J., & Strogatz, S. H. (1998). Collective dynamics of 'small-world' networks. *Nature, 393*(6684), 440–442. https://doi.org/10.1038/30918

Appendix A: Background Experiments

The experiments in this appendix came first. They used smaller graphs where we expected long-range effects to be visible: river gauge networks, synthetic trees, and the Peptides-func molecules of the Long Range Graph Benchmark. They also share the original design, in which a single blend weight α moves a model from message passing ($\alpha = 0$) to attention ($\alpha = 1$).

They taught us four things, and each one fed into the redesign. First, global access helps only when the task needs information that message passing does not deliver; when local information suffices, a hybrid adds only cost. Second, simple descriptors of small graphs, such as how fast the upstream network grows, did not predict the benefit, because there were too few graphs and their size, depth and area moved together. Third, near-tree graphs such as molecules and rivers cannot separate effective resistance from hop distance. Fourth, a mechanistic measurement is only meaningful on a model that has learned the task, and the rebuilt Peptides pipeline did not reach its calibration target. Together these pointed to large graphs with many independent units, a knob that training cannot undo, and tasks whose range is known: the social-graph design of the main text. Table A1 summarises the status of each background experiment. No further work on them is planned.

Table A1. *Status of the background experiments*

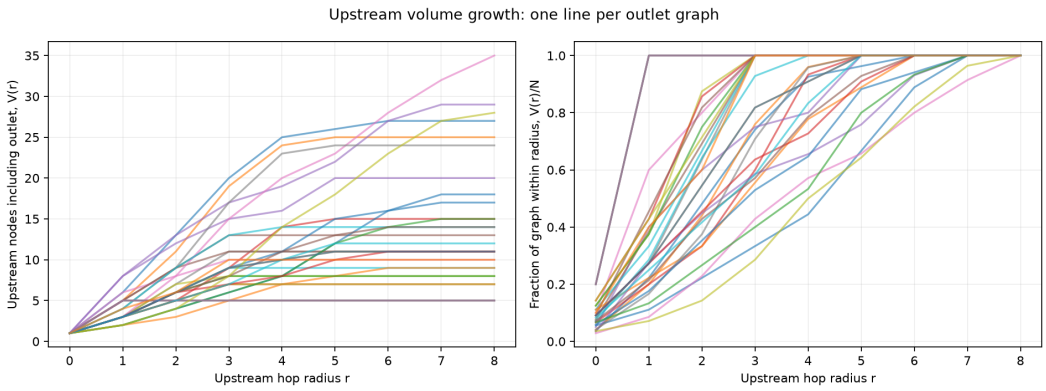
Experiment	Purpose	Status
LamaH-CE river α sweep	Real directed trees, α curve, context ablation	Complete, 72 fits
Controlled tree sweep	Near versus far query across α	Complete, 21 fits, confounded
Pilot Peptides-func replication	GCN and GPS against published values	Pilot, one seed
Pilot mechanistic measurements	Jacobian slope and entropy	Pilot, models did not train
Pilot Reddit experiment	Virtual node versus GraphSAGE by hop	Pilot, unfair baseline
Gate G0 calibration	Rebuilt hybrid against 0.652 AP	Run once; below threshold
Peptides-func re-baseline	AP, Jacobian, entropy, range with CIs	Not run; superseded

River Discharge Forecasting

Setup. The river data come from LamaH-CE (Klingler et al., 2021). For each outlet gauge we built the directed tree of upstream gauges, giving 32 graphs with 5 to 35 nodes and depths of 1 to 8 hops: 27 development outlets and 5 held-out outlets. Each node carries 90 days of daily inputs, and the target is outlet discharge 1, 3 and 7 days ahead. We used two input regimes, weather only and weather plus historical discharge, and split by time (1981–1999 training, 2000–2005 validation, 2006–2017 test). A shared GRU encoded each node's history. Eight spatial layers then combined upstream-to-downstream message passing with attention over a node's upstream ancestors, as $(1-\alpha)\,\mathrm{LN}(\mathrm{MP}) + \alpha\,\mathrm{LN}(\mathrm{Attn})$. Baselines were an outlet-only GRU, topology-free pooling of upstream encodings, and persistence (tomorrow equals today). Error is reported as NRMSE, normalised so that outlets of different size count equally. Rivers are a natural test of reach: when only weather is observed, rain that falls upstream reaches the outlet days later, so the outlet must listen to distant nodes.

We summarised each outlet's upstream structure by its growth profile $V(r)$, the number of upstream gauges within r directed hops including the outlet (Figure A1). Profiles that rise steeply reach many upstream nodes within a few hops.

Figure A1. *Upstream growth profiles $V(r)$ of the river gauge networks*



Results. Table A2 reports held-out test NRMSE, averaged over the five held-out outlets, three horizons and three seeds. With weather inputs only, error fell steadily as α rose from 0 to

0.75, and the $\alpha = 0.75$ blend had 9.6% lower NRMSE than pure message passing. Its advantage over pure attention (0.7%) and upstream pooling (1.1%) was small, so most of the gain came from giving the outlet global access to upstream nodes rather than from blending. With historical discharge added, every $\alpha > 0$ was slightly worse than message passing, and pooling was best. The validation-selected α was stable with weather inputs (0.75, 0.75 and 0.5 at the three horizons) and unstable once flow was supplied (0, 1 and 1).

Table A2. *Held-out river test NRMSE by configuration and input regime (lower is better)*

Configuration	Weather	Weather + flow
Message passing ($\alpha = 0$)	0.2666	0.1691
Hybrid, $\alpha = 0.25$	0.2628	0.1722
Hybrid, $\alpha = 0.50$	0.2489	0.1706
Hybrid, $\alpha = 0.75$	**0.2409**	0.1716
Attention ($\alpha = 1$)	0.2426	0.1713
Upstream pooling	0.2437	**0.1689**
Outlet-only GRU	0.2600	0.1696
Persistence	–	0.1891

Note. Means over five held-out outlets, three horizons and seeds 42–44. Persistence requires flow history and is a single deterministic run.

Figure A2 shows the α curve for each held-out outlet. With weather inputs most outlets improved up to $\alpha \geq 0.5$, but the largest graph (outlet 499, 29 nodes) did not; with flow the curves were flat. Aggregate skill was modest: the mean Nash–Sutcliffe efficiency (Nash & Sutcliffe, 1970) of the weather-only $\alpha = 0.75$ model was -0.30, worse than predicting each outlet's mean flow, against 0.37 for pooling with flow history.

A context ablation tested whether models used upstream information at all (Table A3). Hiding dynamic inputs beyond the outlet raised the error of the $\alpha = 0.75$ blend by about 12% and of pure attention by about 13%, so both used upstream context. Pure message passing did slightly *better* with upstream inputs hidden. A model that is better off without information it could reach is consistent with over-squashing. However, mean aggregation, the attention-only hop bias, layer normalisation and unequal parameter counts could produce the same pattern.

Figure A2. *Held-out test NRMSE against α for each outlet, weather inputs (top) and weather plus flow (bottom)*

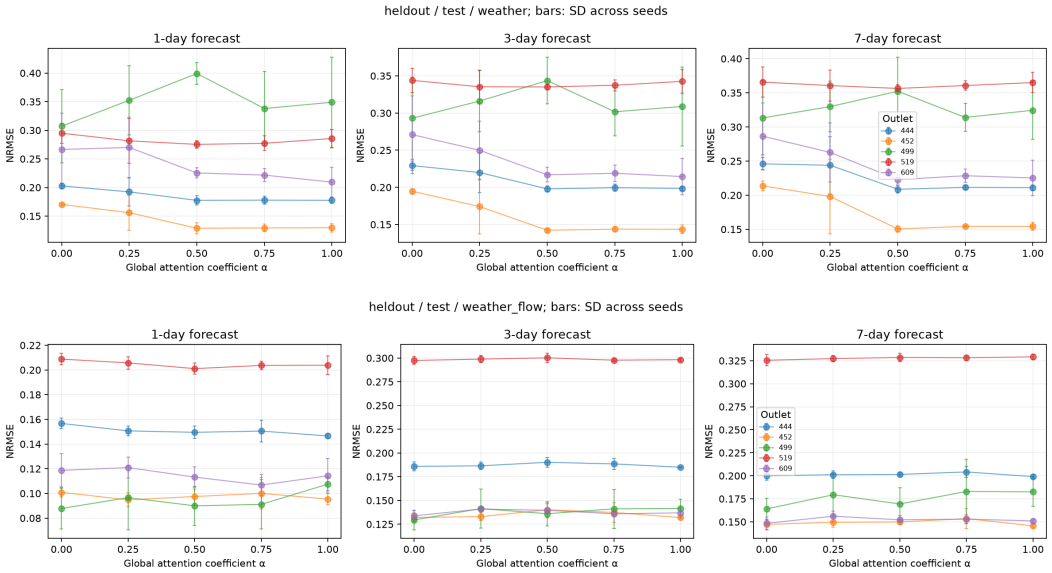


Table A3. *Held-out river test NRMSE under input-context ablation, weather inputs*

Configuration	Full context	Radius 2	Radius 0
Message passing ($\alpha = 0$)	0.2666	0.2673	**0.2584**
Hybrid, $\alpha = 0.75$	**0.2409**	0.2695	0.2741
Attention ($\alpha = 1$)	**0.2426**	0.2583	0.2791

Note. Radius r keeps dynamic inputs only within r hops of the outlet; static attributes and the graph are unchanged. Means over horizons and seeds.

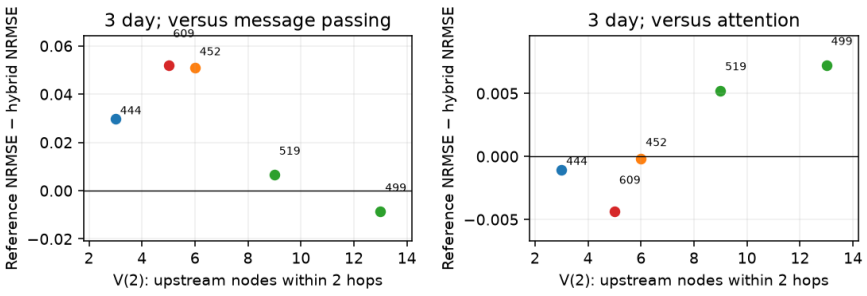
We tested whether upstream growth predicts the hybrid's benefit by relating each held-out outlet's two-hop volume $V(2)$ to the gain of the $\alpha = 0.75$ blend (Table A4 and Figure A3). The fastest-growing outlet, 499, gained least. The Spearman correlation between $V(2)$ and gain was -0.70 at the 1- and 3-day horizons against message passing but $+0.90$ at every horizon against attention, so its sign depended on the reference. Leaving out one dependency group gave correlations from -1.0 to $+0.5$. Across the 27 development outlets, the association was close to zero once size, depth and drainage area were controlled. A graph-conditioned α selector was 2.3% worse than the validation-selected fixed α (macro NRMSE 0.212 against 0.207). Five held-out graphs cannot support a correlation, which is one reason the main study moved to large graphs with many test units.

Table A4. *Gain of the $\alpha = 0.75$ blend over message passing for each held-out outlet*

Outlet	Nodes	Depth	$V(2)$	Weather gain	Weather + flow gain
444	9	6	3	+13.1%	−0.1%
609	7	3	5	+18.8%	+1.3%
452	8	3	6	+26.2%	−2.9%
519	11	3	9	+2.9%	+0.3%
499	29	7	13	−4.4%	−8.9%

Note. Positive values mean the blend has lower NRMSE than message passing. Seeds and horizons averaged first.

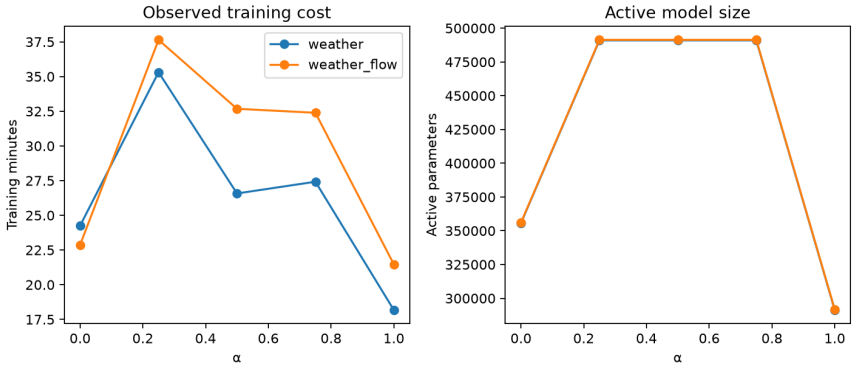
Figure A3. *Two-hop upstream volume against gain of the $\alpha = 0.75$ blend, held-out outlets, weather inputs, 3-day horizon*



The hybrids were also the most expensive models (Figure A4). A blend with both branches had about 491,000 active parameters and took about 80 seconds per epoch, against 53 to 54 seconds

for either endpoint. Upstream pooling came within 1.1% of the best weather-only blend with about 27,000 parameters and 15 seconds per epoch. The river study comprised 72 fits and about 28.5 hours of training.

Figure A4. *Training cost against α for the river models*



Controlled Tree Experiment

Setup. We generated 96 rooted trees with 63 or 127 nodes and a maximum depth of 4 or 8 hops (24 training, 24 validation, 48 test, with no isomorphic duplicates). Every node carries a random standard-normal value. Four nodes at one hop from the root and four at four hops are marked. A query at the root asks for the scaled sum of either the near or the far marked set. Predicting zero gives an MSE of 1, and a predictor that ignores the tree and halves the sum of all eight marked values gives 0.5. The model was the river spatial architecture with eight layers, so both query distances were well inside the reach of message passing.

Results. Message passing answered the one-hop query almost perfectly but failed the four-hop query with an MSE of 1.01, no better than predicting zero (Table A5). Because the far nodes were within its eight-layer reach, the failure is not about reach alone. Every blend solved both queries and beat pure attention on the balanced task; the validation-selected blend was $\alpha = 0.25$. The comparison is confounded by the same architectural differences as the river ablation.

Table A5. *Test MSE on the controlled tree task by query distance*

Configuration	Near (1 hop)	Far (4 hops)	Balanced
Message passing ($\alpha = 0$)	0.0036	1.0094	0.5065
Hybrid, $\alpha = 0.25$	**0.0024**	0.0129	**0.0077**
Hybrid, $\alpha = 0.50$	0.0074	0.0147	0.0110
Hybrid, $\alpha = 0.75$	0.0050	**0.0122**	0.0086
Attention ($\alpha = 1$)	0.0093	0.0189	0.0141

Note. Predicting zero gives MSE 1; a topology-blind predictor gives 0.5. Means over seeds 42–44.

Pilot Results on Peptides-func and Reddit

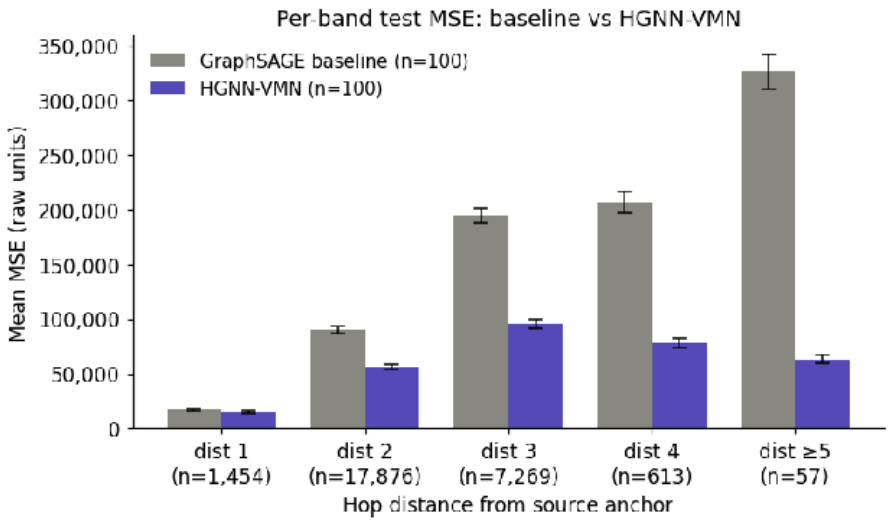
The pilot replication on Peptides-func (Dwivedi et al., 2022) used random-walk structural encodings, 200 epochs with early-stopping patience 40, and one seed. GPS (Rampášek et al., 2022), with four layers of width 96 (386,246 parameters), reached a test AP of 0.6518 (validation 0.6702), matching the published 0.6535. The GCN, with six layers of width 240 (412,790 parameters), reached 0.6263 (validation 0.6545), well short of the tuned value of about 0.686 reported by Tönshoff et al. (2024). The replication therefore reproduced the hybrid but not the tuned message-passing baseline.

The pilot mechanistic notebook reported log-log Jacobian slopes against effective resistance of -1.76 for GINE, -0.48 for the graph transformer and -0.81 for GPS. Those models reached test AP of only 0.54, 0.27 and 0.24, so the attention models had not learned the task. The notebook also trained for only 30 epochs, passed categorical atom codes through a linear layer, used no positional encoding, and averaged attention over heads before computing entropy. Raw entropy rose with graph size for both attention models, but normalised entropy rose only for GPS ($r = +0.60$), not for the transformer ($r = -0.09$). We do not treat these results as evidence; they are shown in Appendix C. These flaws are why the code was rebuilt as the tested `gnn_mech` package.

The pilot Reddit experiment used the SNAP hyperlink graph (Kumar et al., 2018) and compared GraphSAGE (Hamilton et al., 2017) with the same network plus a virtual master node connected to every node. Test nodes were grouped by hop distance from the subreddit with the highest out-degree. Averaged over 100 seeds, the virtual node reduced MSE by 12%, 38%, 51%, 62% and 81% at one, two, three, four and five or more hops (Figure A5). A gain that grows with

distance is what over-squashing predicts, but three flaws prevent that reading. Only the virtual-node model had an input projection layer. The target, character count, was nearly a function of each node's own features. And the distance-based split confounds distance with distribution shift.

Figure A5. *Pilot Reddit test MSE by hop distance from the anchor, GraphSAGE against the virtual-master-node model (100 seeds)*

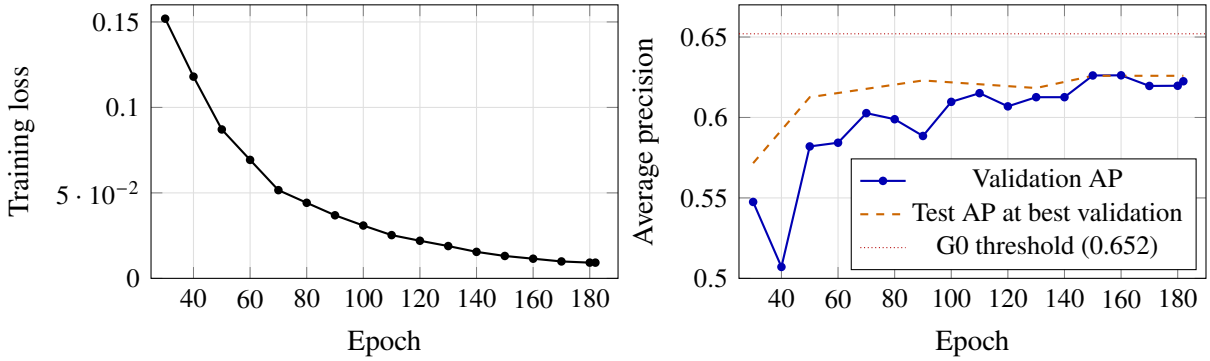


Calibration of the Rebuilt Peptides Pipeline

Before any mechanistic claim on Peptides, the rebuilt pipeline had to pass gate G0: the rebuilt four-layer hybrid at $\alpha = 0.5$ (GINE branch, four-head attention, width 96) had to reach the pilot GPS value of 0.652 test AP on seed 0. On a Colab T4 GPU, training stopped early at epoch 182 after 40 epochs without improvement. The best validation AP was 0.6281 at epoch 142, where test AP was 0.6259, 0.026 below the threshold. Figure A6 shows the training curve. Training loss fell steadily from 0.152 at epoch 30 to 0.009, while validation AP reached about 0.62 by epoch 90 and then stayed within 0.01 of that value. Continued fitting without validation gain indicates overfitting. The run took about 19 seconds per epoch.

The mechanistic measurements for this run did not complete: the graph-level Hessian range exhausted the T4's 15 GB of memory and was stopped. The sanity runs at $\alpha = 0$ and $\alpha = 1$ had not finished, so G0 was not passed. When the project moved to social graphs, we stopped further

Figure A6. *Gate G0 calibration run: training loss and validation AP of the rebuilt four-layer hybrid ($\alpha = 0.5$, seed 0)*



Note. Logged every 10 epochs from epoch 30; the first 20 epochs ran in an earlier Colab session. Test AP is a step function that changes only when validation AP reaches a new best.

Peptides GPU work and kept these results as they stand.

Behaviour of the Measures on Untrained Models

To check that each measure responds to the mechanism it targets, we applied the measures to untrained models built from the same code (Appendix B). The message-passing model's sensitivity fell by a factor of about 3 to 10 per hop and was exactly zero beyond its L-hop receptive field. Pure attention was flat at about $1/N$. The $\alpha = 0.5$ hybrid followed the local curve for two to three hops before settling on a floor about half as high. Attention entropy lay on $\log N$, with normalised entropy of at least 0.995, and the global model's sensitivity to sources six or more hops away fell exactly as $1/N$, from 6.8×10^{-2} at $N = 16$ to 2.1×10^{-3} at $N = 512$. On a 150-node path, the node-level range of the five-layer local model was 0.67 hops, within its receptive field, and grew to 18.7 hops for the hybrid and 34.3 for pure attention. These values describe untrained models and are not findings about trained ones.

Appendix B: Mechanism Checks on Untrained Models

The figures in this appendix use untrained models built from the same code as the trained ones and serve only to show that each measure responds to the mechanism it targets.

Figure B1. *Jacobian sensitivity against hop distance for untrained local, hybrid and global models*

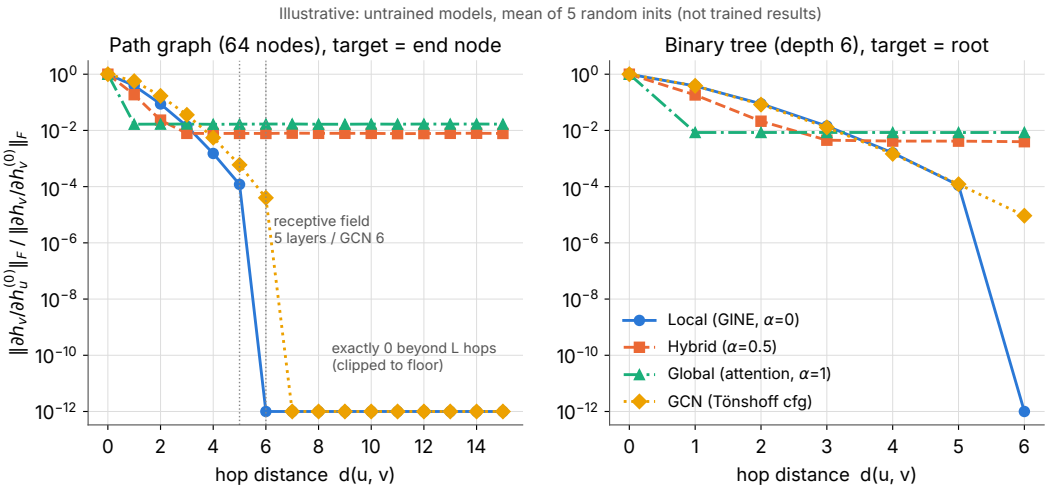


Figure B2. *Attention entropy and long-range sensitivity against graph size for untrained attention models*

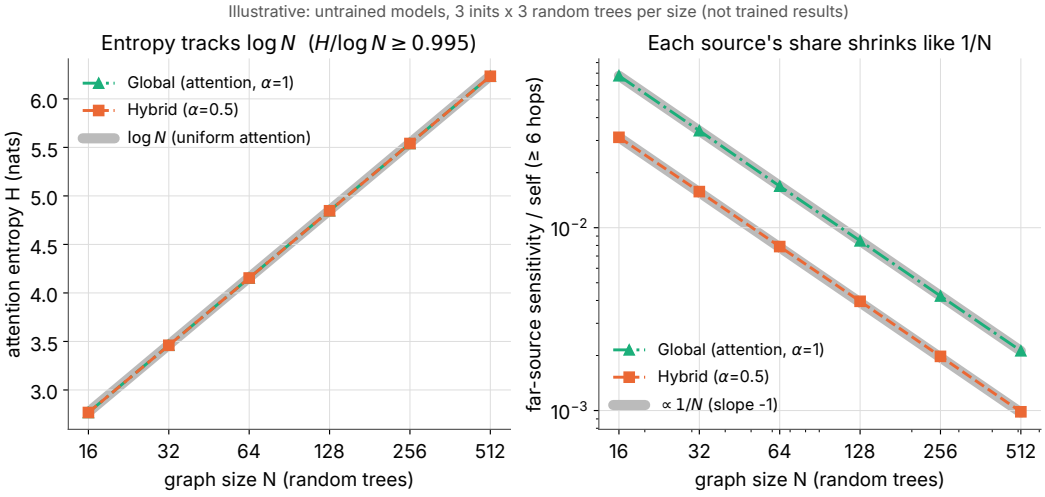


Table B1. *Range measure on a 150-node path graph for untrained models*

Model	Node-level range, hops (mean / max)	Graph-level Hessian range
Local, $\alpha = 0$ (5 layers)	0.67 / 0.69	Undefined (ReLU only)
Hybrid, $\alpha = 0.5$	18.7 / 25.8	Not computed
Global, $\alpha = 1$	34.3 / 47.0	Not computed
GCN (6 layers, GELU)	0.89 / 0.89	17.7

Appendix C: Pilot Mechanistic Results

The pilot results below were produced before the pipeline was rebuilt. Models were trained for 30 epochs on one seed, atom codes were passed through a linear layer, no positional encoding was used, and entropy was computed on head-averaged attention. The test AP of each model is reported because a mechanism measured on a model that has not learned the task is not interpretable.

Table C1. *Pilot Jacobian slopes against effective resistance and test AP*

	GINE	Graph transformer	GPS
Log-log slope b	−1.76	−0.48	−0.81
Correlation r	−0.78	−0.41	−0.69
Test AP	0.54	0.27	0.24

Figure C1. *Pilot Jacobian sensitivity against effective resistance by architecture*

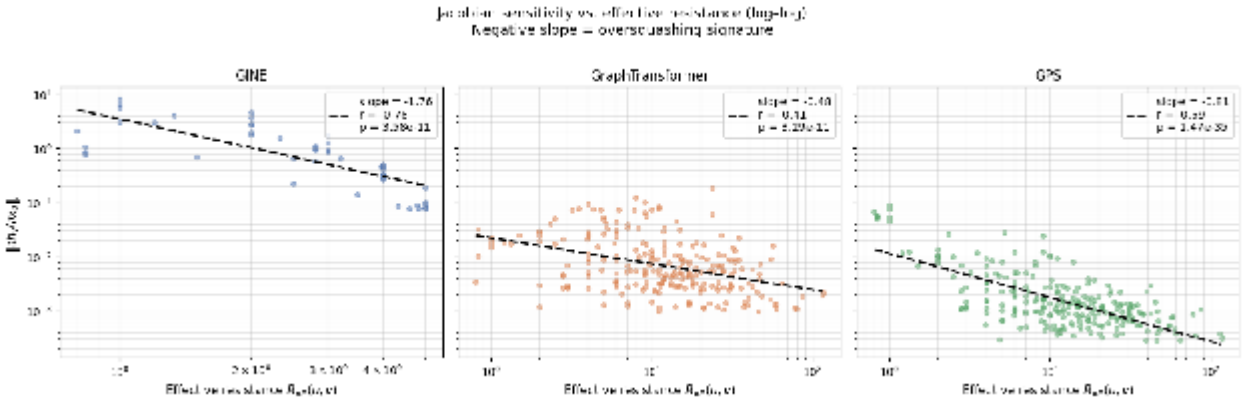
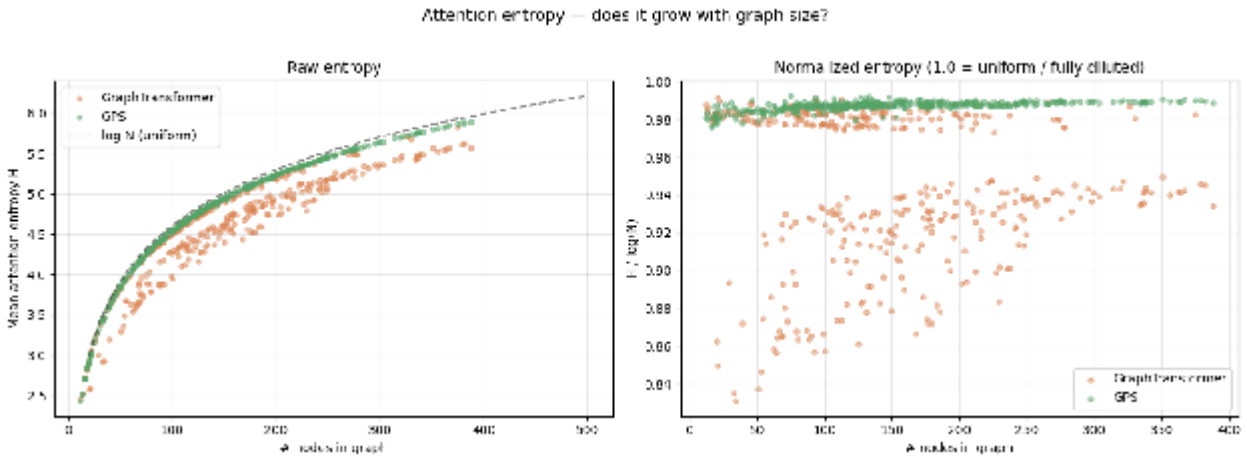


Figure C2. *Pilot raw and normalised attention entropy against graph size*



Appendix D: Model Configurations

Table D1. *Peptides-func model configurations and parameter counts*

Configuration	Hidden	Layers	Heads	Epochs	Parameters
Local, $\alpha = 0$ (GINE)	96	5	–	200	311,055
Hybrid, $\alpha = 0.5$	96	5	4	200	498,255
Global, $\alpha = 1$ (transformer)	96	5	4	200	404,170
GCN (aligned with Tönshoff et al.)	235	6	–	250	491,865
G0 calibration, $\alpha = 0.5$	96	4	4	200	404,654

Note. All models use RWSE with 20 steps, dropout 0.1, AdamW with learning rate 10^{-3} and five warm-up epochs. The benchmark budget is 500,000 parameters.

Table D2. *River model parameter counts and training time per epoch*

Model	Active parameters	Training seconds per epoch
Message passing ($\alpha = 0$)	355,715	52.9
Hybrid ($0 < \alpha < 1$)	490,979	80.1–80.6
Attention ($\alpha = 1$)	291,299	54.2
Upstream pooling	26,627	15.0
Outlet-only GRU	22,403	10.7

Note. Every configuration instantiates 490,979 parameters; the endpoints use only the active branch. Times exclude evaluation, on an RTX 3080 Laptop GPU.

Appendix E: Social-Graph Model and Run Configurations

Table E1 lists the settings of the two completed Higgs sweeps. The first sweep is configured in `configs/social/higgs_follow.yaml`. The affine-free sweep uses `configs/social/higgs_check.yaml` as its base, with the sweep file `configs/sweeps/higgs_affine.yaml`. That file overrides only the target, the split seed, α and the model type. It runs 2 targets $\times$ 3 split seeds $\times$ 7 variants ($\alpha \in \{0, 0.25, 0.5, 0.75, 1\}$, MLP, GCN) $\times$ 2 model seeds, or 84 runs. The subgraph radius sweep (`configs/sweeps/higgs_radius.yaml`) uses the same base with $\alpha = 0.5$, softmax attention masked to $k \in \{0, 1, 2, 3, 4, \infty\}$ hops, and the same targets, seeds and baselines.

Table E1. *Settings of the Higgs follow sweeps*

Setting	First sweep	Affine-free sweep
Subgraph size (METIS parts)	2,000 nodes (12/3/3)	500 nodes (49/11/11)
Validation / test share of parts	0.15 / 0.15	0.15 / 0.15
Split seeds	0	0, 1, 2
Local branch	GINE (sum)	GCN (degree-normalised)
Attention branch	Performer, 64 random features	Exact softmax over the subgraph
Learnable branch-norm scale	Yes	No
Random-walk encoding (RWSE) steps	16	16
Hidden width / layers / heads	64 / 4 / 4	64 / 4 / 4
Dropout / attention dropout	0.1 / 0.1	0.1 / 0.1
Epochs / early-stopping patience	100 / 20	100 / 20
Batch size (subgraphs)	8	8
Learning rate / weight decay	10^{-3} / 0	10^{-3} / 0
Warm-up epochs / gradient clip	5 / 1.0	5 / 1.0
Jacobian sampling	20 graphs, 8 targets, 32 pairs	20 graphs, 8 targets, 32 pairs
Entropy sampling	20 graphs, 64 queries	20 graphs, 64 queries

Note. Both sweeps use the 11 log-scaled inputs (follow in- and out-degree; and, before the cutoff, outgoing and incoming retweet, mention and reply counts and numbers of distinct partners) and AdamW with warm-up and cosine decay. The first sweep ran three seeds on one split; the affine-free sweep ran model seeds 0 and 1 on each split.

The full-graph and planted-target runs follow the build plan in `docs/social_plan.md`. They set `data.sampler` to `full`, so the METIS parts only assign labels to splits while every forward pass uses the whole graph. They train on the full batch, one optimiser step per epoch, and therefore need many more epochs. They use Performer for $k = \infty$ and sparse softmax over the cached

k-hop pair list for finite k. They drop the random-walk encoding (dimension 0), because its exact computation does not scale to the full graph. Planted targets are named `plant:<family>:<k>` and are generated deterministically from the graph, the family, k and a seed. *[Pending: Final full-graph and planted-target configuration files and run counts.]*